

André TONIC

Edward AREVIAN

Philippe MILLET

Jean-Jacques ROUSSEAU

APPLICATIONS EN ASSEMBLEUR DANS L'UNIVERS DU CANON X-07

Reconstruction des pages programmes.

Inclus les correctifs.

RPUFOS 2026

Les auteurs

André TONIC, 19 ans , étudiant en Sciences Economiques, participe activement depuis plusieurs années à l'élaboration de nombreux logiciels commerciaux sur micro-ordinateurs aussi bien portables qu'individuels En outre , il est co-auteur de plusieurs recueils de programmation Il est également président du CLUB C7 et directeur de la publication des Editions NEPTUNE.

Edward AREVIAN, 23 ans, étudiant en électronique et en informatique, est conseiller technique au sein du CLUB C7. Il s'intéresse tout particulièrement à l'architecture interne des micro-ordinateurs Il est également directeur technique des Editions NEPTUNE.

Philippe MILLET , 23 ans , a brillamment terminé ses études commerciales : il est trésorier du CLUB C7 et directeur commercial des Editions NEPTUNE . De plus , il a acquis une certaine expérience dans l'exploitation des langages évolués .

Jean-Jacques ROUSSEAU, 43 ans, est maitre de conférences à l'université du Maine. Il y enseigne la cristallographie et l'électronique. Passionné d'informatique , il est l'auteur de très nombreux logiciels écrits aussi bien en BASIC qu'en ASSEMBLEUR. De plus , il a acquis une très grande connaissance des parties internes du X-07.

Remerciements

Nous désirons adresser nos plus vifs remerciements à :

Mrs De la Rüe du Can et De Carville, responsables MARKETING de la société CANON FRANCE, pour leurs informations relatives au X-07.

Melle HELAS , étudiante , pour sa participation à la composition de cet ouvrage.

Les AUTEURS

Avant-Propos

Composé dans la continuité des "Mystères du X-07", ce deuxième ouvrage complète les connaissances acquises lors de la lecture de ce dernier. Bâti autour de trois grandes parties (Soft, Technique, Applications), ce livre constitue la première référence en matière d'applications pures dans l'univers du X-07 .

Tout en reprenant les recettes qui ont fait le succès des "Mystères" (Taille, clarté, exemples, schémas...), "Applications en ASSEMBLEUR dans l'univers du CANON X-07" apporte son lot de nouveautés avec une présentation originale, un style différent et surtout une pléiade d'applications qui satisferont le plus exigeant des canonistes.

PREFACE

INTRODUCTION

Note de deuxième édition

Code source révisé et corrigé.

Notation :

\$xx : octet en hexadécimal

\$xxxx : adresse d'une routine

xxxxh : adresse d'un emplacement mémoire

Texte amélioré et corrigé.

SOMMAIRE

1^{ère} PARTIE

Où l'on commence par du soft

Utilisation de routines

Nous allons débiter cette première partie par l'un des problèmes les plus importants auquel se heurtent la plupart des programmeurs : l'utilisation des sous-programmes en langage machine sous BASIC.

Comme vous l'avez sans doute remarqué quand vous programmez, l'un des défauts majeurs du BASIC est sa grande lenteur d'exécution. On peut souvent améliorer de façon importante la vitesse d'un logiciel... Comment ?

Tout simplement en écrivant en ASSEMBLEUR les parties critiques du programme !

Deux commandes du BASIC MICROSOFT du X-07 permettent la communication entre les deux langages (BASIC et L.M.).

D'une part, l'instruction "EXEC adresse" permet de lancer une routine écrite en ASSEMBLEUR à partir du BASIC. Notons que cette commande possède une lacune majeure : elle ne permet pas le passage d'arguments.

D'autre part, la directive "USR (adresse, argument)" permet également de lancer un sous-programme écrit en langage machine. L'intérêt majeur de cette commande réside dans le fait qu'elle autorise le passage d'arguments.

Adresse d'implantation d'une routine L.M.

Après chaque arrêt et remise en route normale (sans passer par l'instruction SLEEP) du X-07, les pointeurs de début et de fin de la zone BASIC sont réactualisés. Nous rappelons que le rôle de ces pointeurs et leur utilisation sont décrits dans "les MYSTERES du X-07" : nous ne reviendrons donc pas dessus.

Par conséquent, il est donc possible d'introduire, sans risques importants, des routines écrites en ASSEMBLEUR après le logiciel BASIC.

La méthode classique consiste à introduire les codes dans la zone réservée au moyen d'un ordre CLEAR (Exemple : CLEAR 50,4 000 fixe la limite de la zone utilisateur à l'adresse 4000. Donc, les octets situés après l'adresse 4000 peuvent être utilisés pour écrire des routines L.M.).

La zone réservée est caractérisée par deux pointeurs. MEMSIZ (adresse &H01DF en hexadécimal) représente le pointeur de début (fin de la zone BASIC) et se trouve être initialisé par la commande CLEAR. Le pointeur de fin de cette zone se nomme RAMSTRT (adresse &H210) et fournit le début précis de la zone "fichiers RAM". Ce pointeur est donc actualisé par l'instruction FSET.

Il existe une deuxième méthode pour implanter vos routines. Il suffit simplement d'écrire votre sous-programme dans le fond de la zone des fichiers RAM. Bien évidemment, il faut disposer d'une place suffisante dans cet espace. L'ordre DIR permet d'effectuer ce contrôle.

Si on utilise cette éventualité, il n'est plus nécessaire d'exécuter l'ordre CLEAR à chaque mise sous tension du CANON. Par contre, il est primordial de veiller à ne pas écraser la routine en écrivant dans la zone RAM. De la même façon, vous devez éviter d'implanter votre routine sur un logiciel stocké comme "fichier RAM".

Ces deux méthodes imposent au programmeur de contrôler la gestion de la mémoire et les adresses d'implantation dépendant de la version utilisée (CANON 8Ko, 12Ko, 16Ko, 20Ko, 24Ko... ou plus maintenant grâce aux sociétés de service : pour plus de renseignement à ce sujet, vous pouvez consulter les gazettes du CLUB C7).

Notons qu'il existe d'autres possibilités permettant de s'affranchir de ces difficultés... Mais il est vrai que d'autres problèmes apparaissent ! Nous vous les présentons ci-dessous.

La méthode de l'instruction REM

L'interpréteur ignore ce qui se trouve après une instruction "REM". En effet, quand un logiciel est en cours d'exécution, dès que l'interpréteur découvre "REM", il passe automatiquement à la ligne suivante. Il est donc tout à fait possible d'implanter une routine après REM.

L'utilisation de cette méthode est relativement aisée. Il faut tout d'abord créer une ligne contenant l'instruction REM, suivie d'un nombre de caractères égal à la longueur de la routine. Ensuite, vous pouvez implanter entièrement votre routine par la commande "POKE adresse, code".

Il s'avère très facile de déterminer l'adresse du début d'implantation grâce à l'instruction "RESTORE n° de ligne". En effet, cette directive charge dans les octets &H328 et &H329 l'adresse absolue du début de la ligne en question. Cette adresse est donc égale à :

$$AD = \text{PEEK}(\&H329) * 256 + \text{PEEK}(\&H328) + 6$$

Il faut impérativement rajouter le chiffre 6 afin de tenir compte des octets de chaînage, du numéro de ligne et du code de l'instruction REM (Voir les "MYSTERES du X-07", pages 87 à 90).

Cette possibilité possède plusieurs avantages très intéressants : tous les codes sont admissibles, l'implantation est absolue et invariante, le code est résident. De ce fait, chaque logiciel BASIC apporte ses propres routines L.M. et l'utilisateur n'a pas à s'occuper du chargement de codes supplémentaires.

Par contre, la longueur d'une routine est limitée à 80 codes sur l'écran LCD et à 255 sur l'écran vidéo. De plus, certains codes peuvent "troubler" l'interpréteur : il est donc vivement conseillé de sauter la ligne "truffée" d'une routine au moyen de l'instruction "GOTO". Enfin, l'édition de la ligne, après l'implantation, est impossible et les résultats du listage sont imprévisibles... (Voir l'exemple 1).

La méthode de la chaîne de caractères

Dans le cadre du BASIC MICROSOFT, il existe deux possibilités d'implantation d'une chaîne de caractères...

Dans le corps du programme

Par exemple , la chaîne contenue dans la ligne 10 A\$="TURLUTUTU" possède une adresse fixe étant donné qu'elle réside dans le corps du programme. Cette adresse peut être obtenue par l'intermédiaire de la fonction VARPTR :

$$AD = \text{VARPTR}(A\$) : AD = \text{PEEK} (AD + 2) * 256 + \text{PEEK} (AD + 1)$$

La routine peut alors être implantée grâce à l'instruction "POKE" dans la chaîne de caractères.

Les avantages sont constitués par l'adresse d'implantation absolue et le code résident.

Par contre, les résultats de listage sont totalement imprévisibles. D'autre part, le code 0 est interdit car il indique une fin de ligne. De même, le code &H22 est à bannir : il représente le guillemet, donc une fin de chaîne... Nous vous laissons imaginer les dégâts prévisibles !!

Dans la zone des chaînes

Par exemple, la chaîne résidant dans la ligne 10 A\$ = STRING\$(100,"*") est contenue dans la zone des chaînes de caractères. Son adresse n'est donc pas absolue et peut évoluer tout au long de l'exécution du programme.

Notons que tous les codes peuvent être utilisés et que les problèmes de listage disparaissent.

Le défaut majeur de cette méthode est constitué par le fait qu'elle n'est utilisable qu'avec les programmes relogeables. De plus , il faut impérativement recalculer l'adresse d'implantation avant chaque appel à la routine. D'autre part, il est obligatoire de réimplanter la routine à chaque utilisation (Voir exemple 2).

La méthode du tableau d'entiers

Avec cette possibilité, le code machine est introduit dans un tableau d'entiers (Deux codes pour chaque valeur d'indice) dont on récupère l'adresse en utilisant la fonction VARPTR(T(0)).

Cette méthode est délicate à mettre en œuvre si la longueur de la routine est importante mais elle offre l'avantage de permettre le passage aisé d'arguments multiples. Quelques précautions sont bien évidemment à prendre lors du codage de la routine. En effet, les codes sont introduits deux par deux et sont croisés.

Voici un petit exemple :

Source normal	Source modifié	Code résultant
	NOP	00
LD HL,source	LD HL,source	21 xx xx
	NOP	00
LD DE,destination	LD DE,destination	11 yy yy
	NOP	00
LD BC,longueur	LD BC,longueur	01 zz zz
LDIR	LDIR	ED B0
RET	RET	C9
	NOP	00

Les trois premiers "NOP" servent au cadrage. Le dernier permet d'avoir un nombre pair d'octets.

Voici l'implantation résultante :

Cette méthode a effectivement quelques petits inconvénients : difficulté de mise en œuvre utilisation uniquement avec des codes relogeables évolution de l'adresse d'implantation.

En conclusion, nous pouvons remarquer la relative multiplicité des méthodes d'implantation de routines L.M. Notons qu'après la mise au point du programme, il est possible de supprimer les parties du logiciel ayant servi à l'implantation de la routine (uniquement pour les méthodes 1.1.1 et 1.1.2.1).

Appel de la routine

Nous abordons un secteur très important de la programmation en ASSEMBLEUR : l'appel d'une routine à partir du BASIC. Voici, ci-dessous, les différentes méthodes...

Sans passage d'arguments

L'appel d'une routine sans passage d'arguments constitue le cas le plus simple... Il suffit d'utiliser la fonction "EXEC adresse". Après exécution de la routine L.M., la commande EXEC fait pointer l'interpréteur sur l'instruction suivante. Si la routine ne se termine pas par l'instruction "RET" de l'ASSEMBLEUR Z-80, le CANON se bloque et le "RESET" est nécessaire...

Avec passage d'arguments

Argument unique

Si l'argument est unique on doit utiliser Z = USR (adresse, argument). Deux cas sont alors possibles...

Si l'argument est une chaîne de caractères le registre DE contient l'adresse (ADR) dès l'entrée dans la routine. De plus, l'octet pointé par ADR contient la longueur de la chaîne.

L'adresse physique de la chaîne est contenue dans les deux octets pointés par les adresses ADR + 1 et ADR + 2.

Si l'argument est un nombre, la routine USR transfère la valeur de l'argument dans l'accumulateur (&H44E à &H455) avant d'entrer dans le sous-programme. De plus le registre A contient le type de donnée (2 = entier, 4 = simple précision, 8 = double précision) et le registre HL pointe sur le début de l'accumulateur (&H4 4E).

Pour récupérer l'argument, on peut utiliser les routines de transfert de données (Voir les "MYSTERES...", pages 105-106). Par exemple, la routine CA26 transfère le contenu de l'accumulateur dans la paire de registres BC/DE (cas des nombres en simple précision).

Pour le transfert avec des entiers, vous pouvez utiliser la routine suivante :

```
INC HL
INC HL
PUSH HL          ; sauvegarde de HL pour le retour de la routine
                  (inutile s'il n'y a pas d'argument à retourner au BASIC)
LD E,(HL)
INC HL
LD D,(HL)
EX DE,HL         ; HL contient l'argument entier passé par la fonction USR
```

Pour le retour, la routine USR effectue l'opération inverse : le contenu de l'accumulateur est transféré dans la variable d'appel de USR.

Dans le cas des entiers, vous pouvez passer par l'intermédiaire de la méthode décrite à la page suivante... On suppose que l'entier à retourner est contenu dans le registre BC :

```
POP HL           ; récupère l'adresse des entiers dans l'accumulateur
LD (HL),C
INC HL
LD (HL),B
RET              ; retour au BASIC
```

Arguments multiples

Si vous avez plusieurs arguments à faire passer, la méthode des tableaux est très puissante. Si elle n'est pas utilisable, il faut passer les arguments au moyen d'un tableau dont l'adresse sera l'argument de la routine USR.

Voici un exemple précis. On désire passer les arguments suivants contenus dans le tableau d'entiers :

```
ARG(0) = aa00
ARG(1) = xxyy
ARG(2) = dd00
ARG(3) = zzuu
```

L'appel de la routine se fait par :

```
T = USR (adresse, VARPTR(ARG(0)))
```

Début de la routine :

```
INC HL
INC HL
PUSH HL                ; HL pointe l'adresse de ARG(0)
LD E,(HL)
INC HL
LD D,(HL)
PUSH DE                ; DE = adresse absolue de ARG (0)
POP IX
```

On obtient alors :

```
(IX+0) = aa
(IX+1) = 00
(IX+2) = xx
(IX+3) = yy
```

Par conséquent, l'adressage indexé est utilisé afin de récupérer les valeurs des arguments.

Pour le retour, on peut utiliser la même technique.

Remarquons que dans le cas d'une implantation en adresse absolue de la routine, les arguments seront envoyés par l'instruction POKE avant l'appel de la routine.

Après ces explications théoriques, nous vous avons concocté quelques exemples supplémentaires qui vont vous aider à assimiler les importantes notions que nous venons d'aborder.

Exemples d'utilisation

Exemple N°1

Ce premier exemple va permettre de concrétiser la méthode de l'instruction REM que nous venons de voir (paragraphe 1.1.1).

Remarquons une astuce très intéressante : toute erreur survenant après la ligne 50 provoque le passage automatique dans le mode d'édition LIST @.

Voici le listing du programme :


```

10 GOTO 20
15 REM01234567890123456789
20 RESTORE 15: AD%=PEEK(&H329)*256+PEEK(&H328)+6
30 FOR I%=0 TO 19: READ B$: POKE AD%+I%,VAL("&H"+B$): NEXT I%
40 DATA 06,08,E1,10,FD,2A,15,03,E5,D1,CD,0B,F3,FD,21,00,40,C3,8A,FE
50 ON ERROR GOTO 65000
60 ' DEBUT DU PROGRAMME
100 A = 5 : B = 0 : C = A/B
110 EXEC AD%
64999 END
65000 PRINT "Erreur"
65010 END

```

Exemple N°2

Ce second exemple va démontrer l'efficacité de la méthode de la chaîne de caractères (paragraphe 1.1.2.2).

Voici son listing :

```

10 A$ = STRING$ (20,"#")
20 AD% = VARPTR (A$) : AD% = PEEK (AD% + 1) + 256 * PEEK (AD% + 2)
30 FOR I% = 0 TO 19 : READ B$ : POKE AD% + I% , VAL (" &H" + B$): NEXT I%
40 DATA 06,08,E1,10,FD,2A,15,03,E5,D1,CD,0B,F3,FD,21,00,40,C3,8A,FE
50 ON ERROR GOTO 65000
60 ' DEBUT DU PROGRAMME
100 A = 5 : B = 0 : C = A/B
64999 END
65000 PRINT "Erreur"
65005 AD% = VARPTR (A$) : AD% = PEEK (AD% + 1) + 256 * PEEK (AD% + 2)
65010 EXEC AD%

```

Exemple N°3

La méthode des tableaux d'entiers va être explicitée ci-dessous. Etudiez-la consciencieusement. Bien qu'un peu compliquée, elle pourra vous rendre de nombreux services !!

Voici son listing :

```

10 DIM U%(9)
20 U%(0) = &H0806 : U%(1) = &H10E1 : U%(2) = &H2AFD : U%(3) = &H0315 : U%(4) =
&HD1E5
30 U%(5) = &H0BCD : U%(6) = &HFDF3 : U%(7) = &H0021 : U%(8) = &HC340 : U%(9) =
&HFE8A
50 ON ERROR GOTO 65000
100 D$ = 5
64999 END
65000 PRINT "Erreur"
65010 EXEC VARPTR (U%(0))

```

Exemple N°4

Ce quatrième exemple expose la manière d'écrire une même valeur dans toutes les cases d'un tableau à une dimension.

Si ce tableau est composé d'entiers, on aura $U(5) = N*2$. Par contre, si le tableau est composé de réels en double précision, on aura $U(5) = N*8$.

Vous pouvez comparer la vitesse d'exécution de cette petite routine avec celle d'une boucle écrite en BASIC... Une légère différence transparaît !!

Voici le listing :

```

10 DEFINT A-Z : N = 50 : DIM U(7) , A!(N)
15 AD = 0
20 U(0) = 8448 : U(2) = 4352 : U(4) = 256 : U(6) = -20243 : U(7) = 201
30 A!(0) = 100 : GOSUB 100
40 PRINT A!(0) ; A!(1) ; A!(20)
50 A!(0) = 0 : GOSUB 100
60 PRINT A!(0) ; A!(1) ; A!(20)
70 END
100 U(1) = VARPTR (A!(0)) : U(3) = VARPTR (A!(1)) : U(5) = N*4
110 EXEC VARPTR (U(0)) : RETURN

```

Exemple N°5

Cet avant-dernier exemple vous expose la façon de remplir un écran vidéo très rapidement.

Vous devez ajuster la valeur de la variable US(5) selon le mode écran. Notons que le code à reproduire se trouve en ligne 40.

Voici le listing de cette routine :

```

5 SCREEN 2 : CLS
6 LINE (0,0) - (200,180)
10 DEFINT A-Z : DIM US(7)
20 US(0) = 8448 : US(2) = 4352 : US(4) = 256 : US(6) = -20243 : US(7) = 201
30 US(1) = &H8000 : US(3) = &H8001 : US(5) = 511
40 POKE &H8000 , 64
50 EXEC VARPTR (US(0))
70 END

```

Exemple N°6

Si la fonction "RESTORE n° de ligne" est présente dans le BASIC du CANON, la fonction "RESTORE variable" provoque, par contre, un singulier "UL Error".

Cette fonction peut être très facilement implémentée au moyen de la petite routine relogeable incluse dans le listing ci-joint.

On commence par charger le registre DE avec la valeur de la variable. Ensuite, la routine \$F30D (Voir les "MYSTERES", page 109) recherche le numéro de ligne BASIC et en retour, la valeur de l'adresse de la ligne se trouve dans le registre BC. Notons que si la ligne n'est pas trouvée, le message "UL Error" apparaît tout à fait normalement. Finalement, le pointeur des DATA est actualisé.

Dans le listing ci-dessous, la partie supérieure du logiciel "LOGOGENESE" présent dans le chapitre des applications (en fin d'ouvrage) est reprise. En effet, ce logiciel utilise la technique du RESTORE calculé. Normalement, 33 lignes de DATA sont présentes, des lignes 100 à 132 ainsi que 14 autres, des lignes 200 à 213.

Notons que la routine Z = USR (AD,J) est équivalente à RESTORE J. De plus, la ligne 90 calcule l'adresse d'implantation de la chaîne au moment de l'appel de la routine.

```

1' ATTENTION : NE PAS RENUMEROTER ! !
10 DEFINT A-Z : CLS : PRINT "Logogénèse"
20 RM$ ; STRING$ (18,0) : GOSUB 90
30 FOR I = 0 TO 17 : READ B$ : POKE AD + I, VAL ("&H" + B$) : NEXT I : Z = RND (0)
40 DATA 23 , 23 , 5E , 23 , 56 , CD , 0D , F3 , 60 , 69 , D2 , 38 , F6 , 2B ,
22 , 28 , 03 , C9
45 J = INT (RND (1) * 33) + 100 : GOSUB 90
50 Z = USR (AD,J) : N = INT (RND (1) * 4 + 1)
55 FOR I = 1 TO N : READ D$ : NEXT I
60 K = INT (RND (1) * 14) + 200 : GOSUB 90
65 Z = USR (AD,K) : N = INT (RND (1) * 4 + 1)
70 FOR I = 1 TO N : READ F$ : NEXT I
75 PRINT D$ ; F$ : IF INKEY$ = "" THEN 45
80 IF INKEY$ = "" THEN 80 ELSE 45
90 AD = VARPTR (RM$) : AD = PEEK (AD + 1) + 256 * PEEK (AD + 2) : RETURN
95 END

```

Si vous désirez plus de compléments sur cette technique de "RESTORE calculé", vous pouvez utilement consulter la gazette N°5 du CLUB C7.

Après ces quelques exemples qui, nous l'espérons, auront éclairé quelque peu votre lanterne, nous allons vous exposer tous les codages de variables en mémoire...

En effet, ce problème apparaît comme très peu connu par les utilisateurs du X-07 et pourtant, il apporte un réel complément à la programmation structurée...

Codage des variables

Le codage des variables et des tableaux constitue un passage important à la bonne compréhension des mécanismes internes du X-07. En effet, une bonne partie des informations que vous désirez conserver lors de votre programmation est conservée dans des variables...

Il est donc primordial de bien disséquer ce codage qui, d'ailleurs, n'est pas très évident au premier abord...

Le codage des variables

Le codage des variables peut être appréhendé grâce à la fonction VARPTR du BASIC. Elle affiche l'adresse d'une variable donnée et permet de remonter le codage de cette dernière...

Afin de bien vous faire comprendre le codage des variables dans la mémoire du CANON, nous avons voulu faire passer un message graphique peuplé d'exemples... Reportez-vous à la figure 1.

Voici quelques indications sur la place occupée par les divers types de variables que l'on peut rencontrer :

- Variable entière (%) : 5 octets.
- Variable en simple précision (!) : 7 octets.
- Variable en double précision (#) : 11 octets.
- Chaîne de caractères (\$) : 6 octets + la longueur de la chaîne.

Le codage des tableaux

Les tableaux de variables autorisent de nombreuses applications très pratiques et surtout très puissantes. La figure 2 expose les différents codages possibles.

Voici quelques indications sur la taille en octets qu'ils nécessitent :

- Longueur d'un tableau d'entiers :
 $10 + 2 * \text{indice maximum (une dimension)}$
- Longueur d'un tableau de nombres en Simple Précision :
 $12 + 4 * \text{indice maximum (une dimension)}$
- Longueur d'un tableau d'entiers :
 $10 + 2 * (\text{indice 1 maximum} + 1) * (\text{indice 2 maximum} + 1)$ (deux dimensions)

Quelques exemples d'application :

DIM T%(2 0)	--> Nombre d'octets occupés 50
DIM T!(20)	--> Nombre d'octets occupés 92
DIM T#(20)	--> Nombre d'octets occupés 176
DIM T%(4,5)	--> Nombre d'octets occupés 70
DIM T#(4,5)	--> Nombre d'octets occupés 250
DIM T%(2,4,5)	--> Nombre d'octets occupés 192
DIM T#(2,4,5)	--> Nombre d'octets occupés 732

Nous allons bientôt aborder un sujet passionnant l'utilisation de certaines routines de la ROM.
Alors, accrochez votre ceinture à votre fauteuil car nous allons décoller dans quelques pages !!

CONTENU DE LA MEMOIRE			EXEMPLE D'APPLICATION																														
<table><tr><td>Type (2)</td><td>-3</td></tr><tr><td>NOM 1</td><td>-2</td></tr><tr><td>NOM 2</td><td>-1</td></tr><tr><td>Octet – significatif</td><td>← VARPTR</td></tr><tr><td>Octet + Significatif</td><td></td></tr></table> ENTIERS			Type (2)	-3	NOM 1	-2	NOM 2	-1	Octet – significatif	← VARPTR	Octet + Significatif		En prenant la variable A%=1234, on a :																				
Type (2)	-3																																
NOM 1	-2																																
NOM 2	-1																																
Octet – significatif	← VARPTR																																
Octet + Significatif																																	
			<table><tr><td>02</td><td></td></tr><tr><td>41 (A)</td><td></td></tr><tr><td>00</td><td></td></tr><tr><td>D2</td><td></td></tr><tr><td>04</td><td></td></tr></table>		02		41 (A)		00		D2		04																				
02																																	
41 (A)																																	
00																																	
D2																																	
04																																	
<table><tr><td>Type (4)</td><td>-3</td></tr><tr><td>NOM 1</td><td>-2</td></tr><tr><td>NOM 2</td><td>-1</td></tr><tr><td>Exposant</td><td>← VARPTR</td></tr><tr><td>1^{er} chiffre</td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td>dernier</td><td></td></tr></table> SIMPLE PRECISION			Type (4)	-3	NOM 1	-2	NOM 2	-1	Exposant	← VARPTR	1 ^{er} chiffre							dernier		En prenant CD ! = 1234 = 0,1234 * 10^4 :													
Type (4)	-3																																
NOM 1	-2																																
NOM 2	-1																																
Exposant	← VARPTR																																
1 ^{er} chiffre																																	
	dernier																																
			<table><tr><td>04</td><td></td></tr><tr><td>43 (C)</td><td></td></tr><tr><td>44 (D)</td><td></td></tr><tr><td>44</td><td>Exposant + 40h = 44h</td></tr><tr><td>12</td><td></td></tr><tr><td>34</td><td></td></tr><tr><td>00</td><td></td></tr></table>		04		43 (C)		44 (D)		44	Exposant + 40h = 44h	12		34		00																
04																																	
43 (C)																																	
44 (D)																																	
44	Exposant + 40h = 44h																																
12																																	
34																																	
00																																	
<table><tr><td>Type (8)</td><td>-3</td></tr><tr><td>NOM 1</td><td>-2</td></tr><tr><td>NOM 2</td><td>-1</td></tr><tr><td>Exposant</td><td>← VARPTR</td></tr><tr><td>1^{er} chiffre</td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td>Dernier</td><td></td></tr></table> DOUBLE PRECISION			Type (8)	-3	NOM 1	-2	NOM 2	-1	Exposant	← VARPTR	1 ^{er} chiffre																			Dernier		Prenons PI# = 3.1415926535898 :	
Type (8)	-3																																
NOM 1	-2																																
NOM 2	-1																																
Exposant	← VARPTR																																
1 ^{er} chiffre																																	
	Dernier																																
			<table><tr><td>08</td><td></td></tr><tr><td>50 (P)</td><td></td></tr><tr><td>49 (I)</td><td></td></tr><tr><td>41</td><td></td></tr><tr><td>31</td><td></td></tr><tr><td>41</td><td>Exposant + 40h = 41h</td></tr><tr><td>59</td><td></td></tr><tr><td>26</td><td></td></tr><tr><td>53</td><td></td></tr><tr><td>58</td><td></td></tr><tr><td>98</td><td></td></tr></table>		08		50 (P)		49 (I)		41		31		41	Exposant + 40h = 41h	59		26		53		58		98								
08																																	
50 (P)																																	
49 (I)																																	
41																																	
31																																	
41	Exposant + 40h = 41h																																
59																																	
26																																	
53																																	
58																																	
98																																	
<table><tr><td>Type (3)</td><td>-3</td></tr><tr><td>NOM 1</td><td>-2</td></tr><tr><td>NOM 2</td><td>-1</td></tr><tr><td>Longueur</td><td>← VARPTR</td></tr><tr><td>Adresse LSB</td><td></td></tr><tr><td>Adresse MSB</td><td></td></tr></table> CHAINE			Type (3)	-3	NOM 1	-2	NOM 2	-1	Longueur	← VARPTR	Adresse LSB		Adresse MSB		Prenons la variable T\$ = "TOTO"																		
Type (3)	-3																																
NOM 1	-2																																
NOM 2	-1																																
Longueur	← VARPTR																																
Adresse LSB																																	
Adresse MSB																																	
			<table><tr><td>03</td><td></td></tr><tr><td>54 (T)</td><td>N.B. : Les valeurs</td></tr><tr><td>00</td><td>données dans les</td></tr><tr><td>04</td><td>case sont en</td></tr><tr><td>5B</td><td>hexadécimal.</td></tr><tr><td>05</td><td></td></tr></table>		03		54 (T)	N.B. : Les valeurs	00	données dans les	04	case sont en	5B	hexadécimal.	05																		
03																																	
54 (T)	N.B. : Les valeurs																																
00	données dans les																																
04	case sont en																																
5B	hexadécimal.																																
05																																	

FIGURE 1 : CODAGE DES VARIABLES

Type (2)	-8
NOM 1 (54)	-7
NOM 2 (0)	-6
Offset (19)	-5
Tableau suiv. (00)	-4
Nbre indices (1)	-3
Valeur maxi (0B)	-2
Indice + 1 (00)	-1
T% (0)	← VARPTR
T% (1)	
T% (10)	
Codage de T% (I)	

4 octets		
		1 octet
		2 octets
REPRESENTATIONS		

Type (2)	-12
NOM 1 (41)	-11
NOM 2 (54)	-10
Offset tab. (BB)	-9
Suivant (00)	-8
Nbre indices (3)	-7
Valeur maxi (6)	-6
Indice 3+1 (0)	-5
Valeur maxi (5)	-4
Indice 2+1 (0)	-3
Valeur maxi (3)	-2
Indice 1+1 (0)	-1
T (0,0,0)	← VARPTR
T (1,0,0)	
T (0,1,0)	
T (1,1,0)	
T (2,1,0)	
T (0,2,0)	
T (1,4,5)	
T (2,4,5)	
Codage de AT% (2,4,5)	

FIGURE 2 : CODAGE DES TABLEAUX

FIGURE 3 : CODAGE DES TABLEAUX

Utilisation de routines de la ROM

Ce chapitre est entièrement consacré à l'utilisation de certaines routines de la mémoire morte du X-07. En effet, il est très agréable de posséder une pléiade d'adresses diverses, encore faut-il pouvoir ou savoir les utiliser à l'optimum de leurs possibilités...

C'est en pensant à cet impératif que nous vous avons écrit les quelques routines qui vont suivre. Elles sont très simples à comprendre et vous seront d'une grande utilité dans tous vos logiciels écrits en langage machine.

Evidemment les programmeurs chevronnés trouveront probablement ces petites routines très classiques... Nous les invitons donc à passer au chapitre suivant sans autre forme de procès !

Par contre ceux qui désirent approfondir et optimiser leur programmation vont se délecter... Plusieurs routines de la ROM du CANON ont été utilisées : pour ceux qui ne les connaissent pas reportez-vous aux "MYSTERES du X-07".

D'autre part, les programmes sont présentés sous leur forme "brute". A vous de les implanter où bon vous semble !! Trois champs sont présents pour chaque listing : LABEL, MNEMONIQUE, COMMENTAIRE

Entrons sans plus attendre dans le merveilleux monde de la programmation pure...

La fonction INKEY\$ retrouvée

L'instruction "INKEY\$" se montre indispensable en BASIC. Pourquoi ne pas la créer en langage machine ?... Elle deviendra très vite primordiale : saisie d'options, menu, jeu d'arcade, attente...

Nous allons pour cela, utiliser une routine très pratique située en \$C90A. Rappelons qu'elle permet de saisir un octet au clavier.

Voici le listing de ce petit sous-programme :

INKEY	XOR A	; Attente de la pression d'une touche
	CALL \$C90A	; Appel de la routine
	JR Z, INKEY	; Saut si aucune touche n'est pressée
	RET	; Retour et le registre A contient le code ASCII de la touche pressée

Trace de droites

La routine exposée va vous permettre de tracer un nombre donné de droites sur l'écran LCD du X-07.

Le principe est simple. On utilise le sous-processeur T6834 en lui envoyant les coordonnées de la droite à tracer (4 données indispensables par droite : Xi, Yi, Xf, Yf). Ces nombres sont

contenus dans un espace repéré par le registre HL. Le T6834 trace alors le segment désiré grâce à sa routine N°14 (en hexadécimal).

Dès qu'il a terminé, il recommence si le nombre de droites (contenu dans le registre B) n'est pas nul...

Donc, avant d'utiliser la routine, il faut :

- Remplir une zone tampon avec vos coordonnées
- Charger le registre HL avec l'adresse de début de cette zone
- Charger le registre B avec le nombre de droites à tracer

Voici le listing :

DROITE	PUSH BC	; Sauvegarde de BC (nombre de droites)
	LD A,\$14	; Chargement de A avec la commande 14h
	LD BC,\$0400	; B contient le chiffre 4 (4 paramètres) et C contient 0 car il n'y a pas d'octet de réponse attendue
	CALL \$C92F	; Appel du sous-processeur
	POP BC	; On récupère le nombre de droites ...
	DJNZ DROITE	; ... afin de vérifier si l'on a terminé...
	RET	; ... si oui, on quitte la routine

Remarquons la rapidité de tracé qui n'est pas désagréable. De plus, la routine \$C92F incrémente automatiquement le registre HL... Sublime, non ?!!

Implantation de caractères graphiques

Cette routine va vous permettre de définir des caractères affichables pour les codes ASCII \$80 à \$9F et \$E0 à \$FF.

L'intérêt est de pouvoir définir un nombre de caractères en une seule fois dès que la table est entrée en mémoire.

Le principe est le suivant... Une table des codes est repérée par le registre HL. Cette table doit être structurée de la manière suivante : code du caractère ASCII, 8 codes, code du caractère ASCII, 8 codes, etc...

Le registre B doit être préalablement chargé avec le nombre de caractères à planter.

La routine utilisée (commande N°1A du sous-processeur) se charge d'implanter les codes aux endroits nécessaires...

Voici le listing :

CRECAR	LD B nbre de car.	; Chargement de B avec le nombre de caractères à implanter
	LD HL,TCAR	; Chargement de HL avec l'adresse de début de la table des codes
CC	PUSH BC	; Sauvegarde du nombre de caractères
	LD A,\$1A	; Chargement de A avec le N° de la commande
	LD BC,\$900	; B contient le nombre de paramètres (9) et C le nombre d'octets attendus (0)
	CALL \$C92F	; Appel du sous-processeur
	POP BC	; On récupère le nombre de caractères ...
	DJNZ CC	; ... afin de vérifier si l'on a terminé ...
	RET	; ... si oui , on quitte le sous-programme

Comment tester les curseurs

Tester les curseurs est une chose fréquente quand l'on programme en BASIC. Afin de pouvoir les utiliser aussi sous ASSEMBLEUR, nous vous avons préparé une petite routine très fiable...

Le principe est encore une fois de passer par le T6834 (Nous l'avons pris en affection depuis que nous le connaissons si bien...) grâce à la routine \$C92F.

Notons que les registres A, HL et BC sont modifiés.

FLECHE	CALL \$COBD	; Les tampons clavier et sous-processeur sont vidés
	LD A,\$82	; Commande N°2 du T6834 (STICK)
	LD BC,\$1	; Un paramètre est attendu en retour
	PUSH DE	; Sauvegarde de DE (si nécessaire ...)
	LO DE,BUF	; Le registre DE est chargé avec l'adresse d'un tampon utilisateur que vous devez définir . Ce tampon sert à sauvegarder la réponse du T6834
	CALL \$C92F	; Appel du T6834
	LD A,(DE)	; On charge A avec la réponse contenue dans le Buffer utilisateur
	POP DE	; On récupère DE
	CP \$33	; Curseur droit pressé ? .
	JR Z,DROITE	; si oui , saut à une routine de traitement
	CP \$37	; Curseur gauche pressé ?
	JR Z,GAUCHE	; si oui , saut ...
	CP \$35	; Curseur bas pressé ?
	JR Z,BAS	; si oui , saut ...
	CP \$31	; Curseur haut pressé ?
	JR Z,HAUT	; si oui , saut ...
	RET	; Le registre A contient la valeur \$30 si aucun curseur n'a été pressé ...

Tester une touche particulière

Cette petite routine va vous exposer la manière de tester une touche quelconque du clavier...

Le sous-processeur est encore mis à contribution grâce à sa routine N°28 (en hexadécimal, toujours...). Les registres A, HL, BC et DE sont modifiés après l'emploi de cette routine : n'oubliez pas de les sauvegarder si nécessaire !

N'oubliez pas de charger le registre A avec la touche que vous désirez tester avant de lancer le sous-programme !

Après que la routine se soit exécutée, une touche aura été pressée si le drapeau Z du registre A est à 1. Sinon, il sera normalement à 0...

Voici le listing :

```
TOUCHE    LD HL,BUF+4    ; Le registre HL est chargé avec l'adresse du buffer
           ; utilisateur augmenté de 4
           LD (HL),A      ; Le contenu de A ( code de la touche à tester) est
           ; stocké dans le buffer
           LD A,$28       ; A est chargé avec la commande N°28
           LD BC,$101     ; B contient le nombre de paramètres (1) et
           ; C le nombre d'octets attendus (1)
           LD DE,BUF+2    ; Le registre DE est chargé avec l'adresse du
           ; buffer augmenté de 2 unités
           CALL $C92F     ; Appel du T6834
           LD A,(DE)      ; A est chargé avec la réponse du T6834 stockée dans
           ; le buffer . Si A = 0 , la touche a été pressée
           ; sinon A = $FF
           ORA            ; Les indicateurs Z et S "ressortent" ...
           RET            ; Retour
```

Comment créer une temporisation ?

Enfin une routine permettant la temporisation... C'est normal que tout le monde aime ce genre de programmes : ce sont les seuls permettant de faire une sieste bien méritée !!

Le principe est tellement simple qu'il n'y a rien à expliquer !! Les instructions utilisées sont spécifiques au Z-80 en général donc pas de problème pour les comprendre...

Le registre BC est chargé avec la durée de temporisation désirée... Bon somme !!

Voici le listing :

```
DELAY     DEC B          ; Décrémentation du registre B
           LD A,B         ; Chargement de A avec B pour effectuer une comparaison
           OR C           ; Test...
           JR NZ,DELAY    ; S'il fait encore nuit , on continue à temporiser !!
           RET            ; Le jour se lève : fini de dormir !
```

Musique, Maestro !!

Vous allez enfin pouvoir réaliser votre vieux rêve : réécrire la 9^{ème} symphonie de Beethoven sur CAN ON X-07... Epoustouflant !!

La routine \$C2DF est utilisée pour créer toute sorte de sons : elle est un peu compliquée à utiliser mais les résultats en valent la peine !

Tous les registres sont modifiés. Un seul registre à charger : IX. Il doit impérativement contenir l'adresse de votre mélodie (codée sous formes de notes, bien entendu, dans une zone mémoire). Le tampon est structuré sous cette forme : note, durée, note, durée... Dès que la valeur 255 est trouvée, le X-07 sait qu'il n'y a plus de note ensuite...

Le saxophone est mort... Vive le X-07 ! (Enfin... presque !!)

Mozart en herbe , c'est le moment de prouver vos talents !! Au début de la page suivante, un bel exemple vous est donné...

Exemple de mélodie :

```
AIR1      DB 17, 4, 17, 4, 17, 4, 13, 16, 0, 16, 15, 4
           DB 15, 4, 15, 4, 12, 16, 0, 16, 17, 4, 17, 4
           DB 17, 4, 17, 4, 13, 4, 18, 4, 18, 4, 18, 4
           DB 17, 4, 25, 4, 25, 4, 25, 4, 22, 16, 255
```

Remarque : Si la note est égale à 0, un bruit est généré. D'autre part, n'oubliez pas le terminateur (255).

Fonctions SET, RESET, POINT

Ces fonctions indispensables au "graphiste canoniste" sont très faciles à obtenir via le sous-processeur que vous devez commencer à bien connaître... Après toutes les manipulations faites avec lui, il faut bien reconnaître qu'il facilite beaucoup la vie du programmeur... En effet, vous n'avez guère besoin de connaître toutes les méthodes de travail de l'ensemble des routines !

Il suffit donc d'envoyer le numéro de la commande désirée (Soit \$11 pour SET, \$13 pour POINT et \$12 pour RESET) au T6834 grâce à notre inaltérable routine \$C92F. Avant d'utiliser la routine, choisissez une zone buffer et stockez-y les deux coordonnées du PIXEL (X et Y) sur lequel vous désirez travailler...

Voici le listing :

```

SET      LD A,$11      ; Chargement de A avec la commande $11
          JR SET1      ; Saut à la routine de traitement
POINT    LD A,$13      ; Chargement de A avec la commande $13
          JR SET1      ; Saut ...
RESET    LD A,$12      ; Chargement de A avec la commande $12
SET1     LD HL,BUF     ; Chargement de HL avec l'adresse du buffer utilisateur
          LD BC,$200    ; B contient le nombre de paramètres envoyés (2) et
                        ; C le nombre d'octets attendus (en l'occurrence, 0)
          JP $C92F     ; Le traitement graphique se fait ...

```

Et si l'on parlait d'INPUT ?

Un logiciel informatique a obligatoirement besoin de données pour travailler... Une grande partie de ces données sont encore entrées au clavier et la fonction INPUT joue un rôle déterminant dans ces stockages... Nous vous proposons donc de la recréer !

En effet, la routine va créer la fonction "INPUT" message;"A\$" d'une manière très simple. Quelques fonctions de la ROM sont utilisées telles que \$FEF7 et \$EBF2 : l'une affiche un message et l'autre permet d'obtenir l'entrée clavier. Notons que le message ne doit pas dépasser 20 caractères dans le cas présent.

Voici le listing :

```

ACQMES    LD HL,MESSAGE    ; HL contient l'adresse du message qui doit être
                                impérativement terminé par un 0.
                                CALL $FEF7    ; Le message est affiché...
                                CALL $EBF2    ; L'entrée clavier est obtenue .
                                INC HL        ; On incrémente HL pour obtenir le début du tampon
                                                d'entrée étant donné que la routine $EBF2 fait
                                                pointer HL sur cette adresse -1.

                                LD DE,BUF    ; DE est chargé avec l'adresse du tampon
                                                utilisateur

ACQ        LD C,0          ; C = compteur
                                LD A,(HL)    ; Transfert vers tampon interne
                                LD (DE),A    ; suite du transfert
                                OR A         ; Test du 0 situé en fin de "A$"
                                JR Z,ACQ2    ; Saut à ACQ2 si réalisé
                                INC HL      ; caractère suivant
                                INC DE      ; "
                                INC C       ; incrémentation de C
                                LD A,C      ; Longueur maximale atteinte ?
                                CP 20      ; Test sur la longueur (ici 20)
                                JR C,ACQ    ; Si la longueur est atteinte, on sort de la
                                                boucle...

ACQ2      LD A,C          ;
                                CP 0        ; Si la chaîne est vide, on recommence...
                                JR Z,ACQMES ;
                                LD(LONG),A ; Sauvegarde de la longueur
                                RET         ; Retour

```

Entrée d'un nombre

Nous allons continuer sur notre lancée en vous détaillant une routine équivalente... Cette fois, par contre, vous pourrez entrer uniquement des nombres entiers. La fonction correspond donc à "INPUT" message ";A%".

Notons l'utilisation de routines permettant d'économiser un certain nombre d'octets... En l'occurrence, les routines RST \$10 et \$F595 permettent quantité d'applications : nous vous conseillons de bien les étudier dans "Les MYSTERES du X-07" car elles vous permettront de gagner beaucoup de place en mémoire !

Voici le listing :

```

ACQNUM    LD HL,MESSAGE    ;
                                CALL $FEF7    ; Affichage du message et entrée clavier
                                CALL $EBF2    ;
                                RST $10      ; Teste si un nombre a bien été entré...
                                CALL $F595    ; Transfère le nombre dans le registre DE

```

Utilisation de la X-710

En envoyant une chaîne de codes déterminés à l'imprimante graphique, vous pouvez la contrôler aisément à partir de !'ASSEMBLEUR et lui faire réaliser de très belles choses...

La routine \$CEF7 émet le contenu du registre A vers la X-710. La chaîne de codes doit être, comme à l'accoutumé, stockée dans un buffer.

Voici le listing :

```
LD HL,CHAINE      ; HL est chargé avec l'adresse du début de la
                   chaîne
LPRINT LD A,(HL)   ; A est chargé avec l'un des codes de la chaîne
OR A              ; Test afin de savoir si le dernier code a été
RETZ              ; envoyé vers la X-710
PUSH HL           ; Sauvegarde de l'adresse des codes
CALL $CEF7        ; Transmission du code contenu dans A
POP HL            ; On récupère l'adresse du prochain code
INC HL            ; Incrémentation du registre HL
JR LPRINT         ; On continue le cycle ...
```

Voici un exemple de chaîne à transmettre à la X-710 :

18, 83, 48, 13, 67, 49, 77, 48, 45, 54, 13, 65, 13, 10, 00

```
18 :      mode graphique
83, 48 :   taille "S0"
13 :      retour chariot
67, 49 :   couleur "C1"
77, 48, 45, 54 : commande "M0,-6" --> Remontée de 6 points
13 :      retour chariot
65 :      lettre A (mode texte)
13 :      retour chariot
10 :      commande Line Feed (déroulement du papier)
00 :      fin des commandes
```

Ce chapitre se termine donc sur cette utilisation de notre traceur national, en l'occurrence, la X-710.

Nous osons espérer que nous avons été assez clair... Si c'est bien le cas, vous allez bientôt pouvoir vous précipiter sur les multiples applications situées en fin d'ouvrage.

Mais auparavant, intéressons-nous à un certain crochet d'erreur situé quelque part dans la zone système... c'est fou ce que l'on peut faire avec !!

Extension de fonctions

Introduction

Il existe sur le X-07 plusieurs crochets ("hooks") permettant de prendre le contrôle du CANON dans certaines situations. On peut ainsi modifier les réactions de la machine face à une erreur particulière.

Pour illustrer ce phénomène, nous allons écrire deux routines dérivant ce crochet d'erreur : la première permettra d'obtenir un message d'erreur en français et la deuxième rajoutera une instruction BASIC supplémentaire au X-07. Notons au passage que le CLUB C7 a abondamment parlé de ce crochet dans ses gazettes 2 et 5.

Le principe

Lorsque le CANON rencontre une erreur, il exécute un sous-programme spécial... Mais avant cela, il exécute la routine située à l'adresse ABh. Normalement, on trouve l'instruction RET (code \$C9) à cette adresse... Rien ne nous empêche de placer un saut à cette adresse afin de détourner ce crochet sur l'une de nos petites routines !

Notons que notre tâche se trouve simplifiée car le numéro de l'erreur se trouve dans le registre E du microprocesseur.

Les messages d'erreur en Français

Nous allons donc en premier lieu écrire une routine permettant d'afficher les messages d'erreur en français. Afin de simplifier le programme, nous nous limiterons aux trois erreurs les plus courantes :

- Erreur de syntaxe : code 2
- Erreur de débordement : code 6
- Erreur de numéro de ligne indéfini : code 8

Le travail à réaliser en est d'autant simplifié car traiter tous les cas d'erreur possibles aurait demandé une table assez grande contenant tous les types d'erreurs. Ici, la routine exposée est relativement simple...

Le détournement du crochet

Pour détourner ce crochet, on implante à la place du RET situé en ABh un JUMP XXXX correspondant au début de notre routine personnelle de traitement. Après avoir implanté cette "dérivation", on teste au début du logiciel si le type d'erreur qui survient correspond bien aux erreurs que l'on désire traduire en français... Si c'est effectivement le cas, on continue à exécuter notre routine sinon on revient au crochet (grâce à un RET) et le X-07 affiche l'erreur initiale.

Voici le listing de ce programme :

Afin d'obtenir une présentation plus claire, on fait passer le X-07 à la ligne après l'affichage de chaque type d'erreur et ceci en envoyant les caractères 0A et 0D. L'adresse 1DBh contient le numéro de la dernière ligne exécutée.

Notons que la routine \$F23D place le X-07 en mode "attente-curseur"... Ceci a pour effet d'arrêter le programme pour ne pas retourner au traitement d'erreur originel.

Voici pour terminer cet exemple, le listing BASIC de cette routine :

Une instruction supplémentaire

L'instruction "CARTE"

Nous allons maintenant mettre en place une instruction supplémentaire la commande "CARTE". Sa fonction sera d'engendrer un "Create system ?" permettant de vider la carte.

Le principe de la routine est de détourner le drapeau d'erreur puis de vérifier que l'erreur survenue est bien une erreur de syntaxe. Ensuite, il faut vérifier que cette erreur a bien été engendrée par le mot "CARTE". Si ce n'est pas le cas, on retournera au traitement normal d'erreur .

Comment tester lettre par lettre le mot "CARTE" ? Grâce au registre HL qui pointe sur le caractère suivant l'erreur.

Voici le listing de la routine :

Voici le listing BASIC de cette petite routine :

```

10 ' Instruction CARTE
20 FOR I=&H1C00 TO &H1C00+51
30 READ A$: POKE I,VAL("&H"+A$)
40 NEXT
50 EXEC &H1C00
60 DATA 3E, C3, 32, AB, 0, 21, C, 1C, 22, AC, 0, C9, 7B, FE, 2, C0, CD, 31, 1C,
FE, 45, C0, CD, 31, 1C, FE, 54, C0, CD, 31, 1C, FE, 52, C0, CD, 31, 1C, FE, 41,
C0, CD, 31, 1C, FE, 43, C0, C3, D1, C4, 2B, 7E, C9

```

Instruction "PAINT" détournée

Pour terminer ce chapitre, voici une petite routine rajoutant l'instruction "PAINT" au BASIC du X -07. Cette commande aura uniquement pour fonction d'afficher le message "PAINT" à l'écran.

Le drapeau de "PAINT" (instruction utilisée en vidéo avec la X-720) se situe en 99h. Nous allons procéder de la même manière que pour le détournement des erreurs en plaçant à cette adresse un JP XXXX nous envoyant à l'adresse de notre routine...

Voici le listing de ce programme

```

5 ' [ ; Début du programme
10 ' ORG $1C00 ; Les lignes 10 à 70 détourne le
30 ' LD A.$C3 ; crochet d'erreur sur notre routine.
40 ' LD ($99).A
50 ' LD HL.#DB
60 ' LD ($9A).HL
70 ' RET
80 ' #DB PUSH HL ; les lignes 80 à 110 affichent le
90 ' LD HL.#ME ; message...
100 ' CALL $D5B0
110 ' POP HL
120 ' RET ; On continue le programme
130 ' #ME DEFM PAINT
140 ' DEFB $00
150 ' ]

```

Les applications de ces crochets d'erreur sont multiples comme vous avez pu le constater... Nous allons maintenant nous plonger dans la partie "HARD" de cet ouvrage...

2^{ème} PARTIE

... Où l'on continue par du HARD ...

Entrées/sorties & interruptions

Ce chapitre sur les entrées/sorties et les interruptions constitue la quintessence du CANON... En effet , le X-07 est très réputé pour la multiplicité de ses entrées/sorties et nous avons pensé nous aventurer un peu sur ce terrain relativement inexploré...

Notons que le CLUB C7 a édité un très bon dossier ("Le X -07 s 'évade") sur le sujet : notre machine adorée se branche alors sur une lampe, un réveil, un magnétophone... et les commande remarquablement !! Avis aux amateur aimant les sensations fortes !!

Les interruptions

Généralités

Les interruptions constituent des méthodes d'entrées/sorties un peu particulières. En effet, les entrées/sorties programmées accusent deux inconvénients majeurs :

- Elles font perdre du temps au microprocesseur étant donné qu'il doit examiner l'état de tous les périphériques.
- Après avoir déterminé le périphérique devant entrer en contact avec lui, le Z-80 exécute le programme correspondant. Malheureusement, ce temps de réponse peut devenir critique si un périphérique véloce a besoin ce processeur... C'est le cas ici avec l'écran ou les entrées/sorties sérielles.

Une solution a été découverte en reliant chaque périphérique à une ligne d 'interruption. Le Z-80 examine à la fin de chaque cycle-machine (instruction) ces lignes d 'interruption. Si un arrêt est demandé le microprocesseur sauvegarde l'adresse à laquelle il travaillait afin d'exécuter la routine correspondant à l'interruption demandée.

De plus, quand une interruption est en cours, il ne faut peut-être pas qu'une autre interruption l'interrompe... Pour éviter cela, il est possible d'interdire les interruptions en leur donnant un niveau de priorité différent.

A ce moment-là, l'interruption possédant le plus haut degré de priorité ne pourra être stoppée par aucune autre, mais, par contre, cette interruption pourra interrompre toutes les autres !!

Les types d'interruption

Le CANON X-07 - est doté de deux types distincts d'interruptions :

Les interruptions A, B, C ayant chacune un niveau défini de responsabilités.

- INT A : interruption provenant du sous-processeur T6834. Le programme d'interruption correspondant se situe en C799h.
- INT B : interruption sérielle . Un saut est provoqué à l'adresse C7A3h.

- INT C : cette interruption n'est utilisée que quand le X-07 est relié à la X-720. Une horloge reliée à cette ligne d'interruption oblige le CANON à afficher le curseur.

L'interruption NMI n'est pas utilisée sur le X-07. C'est l'interruption possédant le plus haut niveau de priorité... En effet, NMI se traduit en anglais par "No Masquable Interrupt" ce qui signifie bien évidemment "interruption non masquable". Quel que soit la tâche effectuée par le X-07, il suffit qu'un signal d'interruption parvienne sur la ligne NMI pour que le CANON exécute immédiatement le programme correspondant. Notons que cette interruption très particulière va nous être très utile...

Le traitement des interruptions

Une fois l'interruption acceptée, il faut impérativement satisfaire "le demandeur" sans perdre le fil du programme en cours. Ceci implique une sauvegarde des registres dans la pile, le minimum étant de sauver le compteur ordinal (PC) afin de pouvoir y placer l'adresse de la routine d'interruption à exécuter.

Après avoir effectué le traitement demandé par le périphérique, le microprocesseur peut reprendre tranquillement le fil de son autre programme au point où il l'avait quitté grâce à l'adresse sauvegardée sur la pile (correspondant à PC). La figure exposée à la page suivante vous permettra de bien assimiler le processus.

Les interruptions peuvent être déconnectées ou reconnectées à l'aide de deux ordres du NSC 800 : EI et DI.

FIGURE 4 : LE TRAITEMENT DES INTERRUPTIONS

En traduisant ces deux abréviations on obtient Enable Interrupt (interruption autorisée) et Disable Interrupt (interruption interdite).

En général, on interdit les interruptions lorsque l'on commence le traitement d'une interruption de faible priorité afin de ne pas être dérangé par un autre arrêt. Mais n'oubliez surtout pas de tout remettre en ordre dès que la partie critique de votre routine est terminée grâce à l'instruction EI.

L'utilisation des interruptions sur le X-07

Le fonctionnement du X-07 est intimement lié à un sous-processeur nommé très poétiquement "T6834 " Nous en avons abondamment parlé depuis le début de cet ouvrage et nous allons utiliser la ligne d'interruption B afin de détecter les interruptions provenant de ce sous-processeur. Par exemple, un appui sur la touche BREAK ou OFF constitue une interruption.

En fait, les interruptions provoquent des sauts à quatre adresses différentes suivant le type d'arrêt demandé. Ainsi :

- INT A provoque un saut en 3Ch
- INT B provoque un saut en 34h
- INT C provoque un saut en 2Ch
- NMI provoque un saut en 66h

On peut donc écrire à ces adresses soit le programme que l'on désire faire effectuer au X-07, soit un "JP XXXX" afin que le Z-80 aille exécuter le programme se trouvant à cette adresse.

Sur le CANON, cette deuxième solution a été prônée et l'on trouve logiquement JP C7A3, JP C799, etc...

Cette solution va nous rendre un grand service étant donné que ces adresses se trouvent en RAM... Nous allons donc pouvoir les modifier à volonté et découvrir véritablement ce qui s'y passe !

En reprenant l'exemple du T6834, nous nous rappelons que ce dernier envoie un signal sur l'une des pattes du NSC800 lorsqu'il désire communiquer. Si l'interruption est prise en compte, le microprocesseur ira effectuer la routine siégeant à l'adresse 003Ch. Or, l'adresse 003Ch contient l'instruction de saut JP \$C799... Par conséquent, le programme d'interruption continuera sa course à l'adresse C799h !

De même , si une interruption parvient sur la patte NMI du NSC800, ce dernier exécutera les instructions situées à l'adresse 66h... Mais, notre bon vieux processeur trouvera la directive RET (code \$C9) à cet endroit et reprendra l'exécution normale du logiciel étant donné que l'interruption NMI n'est pas utilisée sur le CANON. Justement, nous allons détourner cette interruption et jouer un peu avec les entrées/sorties...

Applications pratiques

Les interruptions étant généralement déclenchées par des périphériques, ce sont des pattes spécialisées du microprocesseur qui se chargent de leur détection. Nous allons essayer de les utiliser afin de bien comprendre ce qui se passe lors d'un arrêt demandé.

Pour cela, il nous faut un peu de matériel... Par exemple , un doigt pour retirer le couvercle du bus (prise située à l'arrière du X-07 : port d'extension), une paire de lunettes pour lire le brochage (page 127 du guide de l'utilisateur) et un petit morceau de fil électrique pour nous livrer à quelques liaisons coupables !

L'expérience est relativement simple. Les bornes correspondant aux interruptions A et NMI se trouvent sur le connecteur arrière du X-07. Après avoir changé les adresses de saut d'une de ces routines , nous provoquerons un saut en reliant la broche correspondante au +5 volts ou au 0 volt , selon la tension requise.

Pour cette expérimentation, nous utiliserons l'interruption NMI. En effet, cette borne est sensible au niveau haut (+5V) alors que INT A est sensible au niveau bas (0V), donc plus "sensible" électriquement. Le brochage du connecteur est décrit à la page suivante : la broche NMI correspond à la broche 20; le +5V est présent sur les broches 39 et 40.

Dès que nous désirerons créer notre interruption, il nous suffira d 'un "bref" contact entre ces deux bornes.

La première expérience

Nous allons mixer un peu de BASIC et de langage machine uniquement pour une application amusante.

Le but de la routine sera de faire compter le X-07 de 0 à 10000... De plus, dès qu'une interruption se produira, il devra émettre un BEEP sans s'arrêter de compter !! Enfin du multitâche sur X-07 !!

En fait, la routine va être composée de deux petits programmes :

- Le premier va se charger de modifier l'adresse de saut du NMI afin que le X-07 puisse exécuter notre programme dès que nous relierons nos deux bornes...
- Le deuxième s'occupera d'émettre un BEEP. Pour cela, l'écriture du caractère "BELL" suffit (code ASCII N°7).

Le programme écrit en langage machine est lui aussi très simple. Ceux qui ne possèdent pas d'ASSEMBLEUR pourront se servir du petit programme BASIC joint aux listings.

Dès que le message "court-circuitez les broches" s'affiche, vous devez relier les bornes 20 et 39 (ou 40) du connecteur d'extension. Un simple toucher devrait être suffisant.

Vous entendrez (à condition de ne pas avoir coupé le buzzer) distinctement le BEEP avec un compteur défilant !

Vous pouvez remarquer que la méthode de changement d'adresse des interruptions ne diffère pas du changement des « Hooks » lors d'une modification du traitement des erreurs.

Un deuxième programme

Ce deuxième exemple va traiter d'un problème très particulier : l'interaction de deux routines écrites en langage machine.

Nous allons interrompre le traitement d'une routine LM afin d'en effectuer une autre et dès cette deuxième routine terminée, nous reprendrons le cours de la première jusqu'à la prochaine interruption ! Tout ceci va être réalisé grâce à notre fidèle petit fil électrique.

Le programme va afficher à l'écran tous les codes ASCII compris entre 0 et 255.

Lorsque vous court-circuiterez les broches 39 et 20 un message s'affichera mais le défilement ne s'arrêtera pas pour autant.

Pour des raisons de simplification, les codes affichés s'échelonnent de 0 à 255. Par conséquent, ne vous étonnez pas si aucun caractère n'apparaît durant les premières secondes ! De plus, une temporisation située entre les lignes 215 et 320 du listing source permet de ralentir l'affichage des caractères. Si elle n'était pas présente, vous ne pourriez rien distinguer à l'afficheur tellement la routine est véloce ! Cette routine est d'ailleurs utilisée dans le deuxième programme pour que le message puisse rester visible à l'écran suffisamment longtemps.

Nous conseillons aux amateurs de se procurer un connecteur correspondant à la prise d'extension du X-07. Ensuite vous pourrez câbler un petit bouton poussoir entre les bornes 39 et 20... Le court-circuit à effectuer en sera d'autant facilité car vous pourrez remarquer qu'un contact trop prolongé entre les deux bornes peut fausser le bon déroulement des opérations.

FIGURE 5 : LES DIFFÉRENT BROCHAGES

Pour ceux qui ne possèdent pas d'ASSEMBLEUR, le logiciel BASIC charge les deux parties du programme : la première détourne les interruptions et la deuxième est constituée par les deux petites routines.

Dès que le premier programme est chargé en mémoire, vous pouvez commencer à envoyer des interruptions au X-07.

Les bricoleurs chevronnés pourront envisager par exemple de construire une petite horloge qui générerait un "top" toutes les "X-ièmes" de secondes. Ils auront la possibilité de simuler un fonctionnement en multitâche du X-07 !

Pour cela, il suffit de placer à l'adresse de l'interruption le deuxième programme que l'on désire exécuter. Une solution intéressante consiste à récupérer le signal CLOCK présent sur la borne 29 et de le diviser à l'aide de bascules successives ou de compteurs afin d'obtenir un signal de fréquence moindre (de l'ordre de 10 à 5). Ce signal sera alors à renvoyer sur la broche 20 du connecteur.

Pour ceux qui désirent améliorer le système de bouton-poussoir, le problème des effets de retour peut être résolu en introduisant une bascule dans le système.

Conclusion

La détection d'interventions extérieures ouvre un champ d'applications gigantesque au programmeur : détection de niveaux, surveillance d'un appartement, détection d'une sonnerie téléphonique en vue de l'édification d'un serveur, etc...

L'emploi des interruptions reste un moyen relativement simple pouvant être dédié à la résolution des problèmes liés à l'interfaçage des périphériques rapides. On peut noter que l'interruption NMI est très utilisée dans le but de relier le microprocesseur à un lecteur de disquettes ou à un circuit d'accès direct en mémoire (DMA).

Les personnes ne se sentant pas une âme bricoleuse peuvent quand même simuler le fonctionnement multitâche en utilisant l'interface vidéo. Mais attention ! La durée du programme d'interruption est critique et si la X-720 envoie une impulsion avant que le programme ne soit terminé, le RESET général est à prévoir.

```

5 REM 2 PRG EN LM
10 '['
20 ' ORG $1A00
30 '*****
40 '**          Chent du saut du NMI          *
50 '*****
60 'LD A.$C3
70 'LD ($66).A
80 'LD HL.#DB : Debut du 2eme prog
90 'LD($67).HL
100 'RET
110 '*****
120 '**          le 1er programme          *
130 '*****
140 'ORG $1B00
150 'LD A.$00
160 '#B1 INC A
170 'LD HL.$0502
180 'LD ($B8).HL
190 'CALL $C1BE
200 'CALL #TP
210 'JR #B1
215 ' ** TEMPO **
220 '#TP PUSH AF
230 'PUSH BC:
240 'LD A.$7F
2s0 '#B2 LD B,$FF
260 '#B3 DEC B
270 'JR NZ.#B3
280 'DEC A
290 'JR NZ.#B2
300 'POP BC
310 'POP AF
320 'RET
330 '*****
340 '**          le 2eme programme          *
350 '*****
360 '#DB PUSH AF
370 'PUSH BC
380 'PUSH DE
390 'PUSH HL
400 'LD HL.$0101
410 'LD ($88).HL
420 'LD HL.#ME
430 'CALL $FEF7
440 'CALL #TP

```

FIGURE 6 : LES 2 PROGRAMMES

```

450 'CALL $CE9E
460 'POP HL
470 'POP DE
480 'POP BC
490 'POP AF
500 'RETI
510 '#ME DEFM INTERRUPTION !!!!
520 'DEFB $00
530 ']

```

```

10 REM LES 2
15 RESTORE 500
20 FOR I=&H1A00 TO &H1A0B
30 READ A$: POKE I,VAL("&H"+A$): NEXT
50 RESTORE 700
60 FOR I=&H1B00 TO &H1B1F
70 READ A$: POKE I,VAL("&H"+A$): NEXT
80 RESTORE 800
90 FOR I=&H1800 TO &H182E
100 READ A$: POKE I,VAL("&H"+A$): NEXT
110 EXEC &H1A00: EXEC &H1800
500 DATA 3E,C3,32,66,00,21,00,18
501 DATA 22,67,00,C9
502 REM P1
700 DATA 3E,00,3C,21,02,05,22,B8
701 DATA 00,CD,BE,C1,CD,11,1B,18
702 DATA F1,F5,C5,3E,7F,06,FF,05
703 DATA 20,FD,3D,20,F8,C1,F1,C9
704 REM P2
800 DATA F5,C5,D5,E5,21,01,01,22,B8,00
801 DATA 21,1C,18,CD,F7,FE,CD,11,1B,CD
802 DATA 9E,CE,E1,D1,C1,F1,ED,4D
803 REM Message
804 DATA 49,4E,54,45,52,55,50,54,49,4F
805 DATA 4E,20,21,21,21,21,00

```

FIGURE 6 : FIN

```

10 '['
20 ' ORG $1800
30 '*****
40 '**      CHent du saut pour NMI      *
50 '*****
60 'LD A.$C3
70 'LD ($66).A
80 'LD HL.#DB
90 'LD ($67).HL
100 'RET
110 '*****
120 '**      LA ROUTINE DE BEEP      *
130 '*****
140 '#DB LD A.$07:*"BELL=CHR$(07)
150 'CALL $C1BE
160 'RETI
170 ']'

```

FIGURE 7 : LE BEEP !!

```

5 REM LE BEEP
10 FOR I=&H1B00 TO &H1B12
20 READ A$:POKE I,VAL("&H"+A$)
30 NEXT
40 CLS: PRINT"COURT-CIRCUITEZ LES
BROCHES"
50 FOR I=1 TO 10000
60 LOCATE 10,2: PRINT I
70 NEXT
100 DATA 3E,C3,32,66,00,21,0,15
101 DATA 22,67,0,C9
102 DATA 3E,7,CD,BE,C1,ED,40

```

FIGURE 7 : FIN

Les entrées/sorties du X-07

Une des particularités du CANON est représentée par son exceptionnelle ouverture vers le monde extérieur. En effet, quatre prises très importantes trônent sur trois des côtés de notre machine : la prise magnétophone, série, parallèle et la prise d'extension.

On peut facilement utiliser le port d'extension du bus du X-07 car l'Assembleur Z-80 comprend parfaitement les ordres IN et OUT permettant d'utiliser cette prise.

Par contre, les prises série et magnétophone sont beaucoup plus mal loties. Leur programmation passe en effet par l'ACIA et nécessite de ce fait une très bonne connaissance de l'organisation interne du X-07.

Il existe pourtant une solution intéressante pour simplifier l'utilisation de ces différents ports : il suffit d'employer les routines ROM correspondant à la fonction INIT #n afin d'ouvrir un fichier d'entrées/sorties. Dans ces conditions, plus rien ne s'oppose à une utilisation très performante du X-07 en matière d'entrées/sorties.

En effet, l'utilisation simultanée de la prise série et du Langage Machine va permettre de réaliser quantité d'applications originales.

Tenez-vous prêt !

Les divers dispositifs

Avant de passer en revue les divers dispositifs d'entrées/sorties existant sur le X-07, notons que si l'entrée de mots se fait toujours à une vitesse fixée par le dispositif relié au X-07, la durée entre l'entrée de deux mots peut être considérablement réduite...

Les dispositifs assignables par l'ordre INIT #n

Il existe cinq dispositifs directement assignables par l'ordre INIT #n :

- GPR : ce dispositif correspond à l'imprimante graphique . Lorsqu'il est initialisé, la taille des caractères est fixée à 2 et les codes "retour chariot/fin de ligne" sont envoyés à la X-710.
- LPT : ici, seuls les codes "retour chariot/fin de ligne" sont envoyés sur la prise parallèle (Centronics).
- KBD : la mémoire tampon du clavier est remise à 0.
- MEV : le fichier dont le nom est spécifié est recherché. S'il existe, les pointeurs de lecture et d'écriture sont placés au début du fichier La taille étant alors ignorée, il est donc inutile de la spécifier. Par contre , si le fichier n'existe pas, il est créé avec la taille spécifiée.
- CON : ce dispositif ne fait strictement rien.

Les autres dispositifs

Ces dispositifs sont au nombre de cinq : CASI , CASO , OPT , COM , PRT.

Dans ces différents cas, on assigne à une des sorties spécifiées le dispositif d'entrées/sorties désiré. On règle le générateur de Bauds à la vitesse désirée ainsi que le mode de transmission.

Notons que toutes ces fonctions ne sont réalisées que pour le dispositif COM. En effet, les réglages de cadence de transmission et de mode se font automatiquement pour les quatre autres systèmes.

L'initialisation

Tous les dispositifs sont initialisés de la même façon à l'aide de l'appel à un sous-programme unique. Il faut impérativement spécifier lors d'un appel le dispositif que l'on désire utiliser.

Lors de l'appel à ce sous-programme, le X-07 va rechercher dans une table à quel dispositif il doit accéder. Dès qu'il a résolu cette première phase, il programme ses interfaces de la manière demandée. Notons qu'une recherche dans la mémoire morte permet de retrouver les noms des dispositifs initialisables.

Cette table constitue un catalogue des systèmes existants. Prenons l'exemple de "CASI"; à l'adresse E782h, nous allons trouver en hexadécimal les données suivantes :

43 41 53 49 BA 00 02 12 E5 0A E0 35 DE 0F E1

C'est en précisant l'adresse du numéro du dispositif que nous pouvons spécifier le dispositif que l'on désire mettre en œuvre lors de l'appel au sous-programme d'initialisation commun à tous les systèmes.

Pour cela, nous devons utiliser une partie spécialisée de la zone système du X-07 située à l'adresse 02C5h. Lors de l'ouverture du dispositif, on emploiera le registre DE afin de pointer sur le numéro du dispositif à initialiser. Le X-07 placera alors à l'adresse 02C5h l'adresse du dispositif dans le tableau et sauvegardera les registres HL, BC et DE à chaque fois que l'on fera appel au système en question.

Notons que cinq dispositifs peuvent être ouverts au maximum. En effet, la zone allant de 02C5h à 02E8h ne peut contenir que 40 octets : deux octets pour l'adresse du tableau et six octets pour les trois registres à sauvegarder et ceci, cinq fois seulement. Voilà pourquoi on ne peut utiliser un numéro de fichier supérieur à 5.

La table en ROM est codée de cette manière :

Nom du dispositif .
Numéro du dispositif
Deux octets pour le sous-programme de sortie d'un octet
Deux octets pour le sous-programme d'entrée d'un octet
Deux octets pour le sous-programme d'initialisation
Quelques octets sont laissés pour un sous-programme optionnel

Après avoir passé en revue tous les dispositifs existants, nous allons pouvoir entrer dans l'étude de routines d'entrées/sorties.

Les fichiers CASO & CASI

Nous allons réaliser une petite routine de sauvegarde et de récupération de données sur K7. Pour réaliser cette application, nous avons besoin de plusieurs choses :

- Initialiser le fichier de sortie pour la sauvegarde
- Déterminer un "délimiteur" qui nous indiquera le début et la fin de chaque donnée
- Initialiser le fichier d'entrée pour recharger les données
- Un test vérifiant que nous sommes bien en présence du début des données
- Une mémoire tampon nécessaire à la sauvegarde des données lues
- Un test vérifiant que nous sommes bien en présence de la fin des données

De plus, nous devons dans ce cas signaler une erreur d'entrée si un problème survenait. Nous utiliserons dans ce but les indicateurs du registre F (C et F) nous indiquant la présence d'une erreur.

Enfin, nous devons tenir compte du fait que la routine ROM d'entrée de données n'attend pas les données : une boucle de temporisation est donc fortement conseillée.

Le programme de sortie des données

L'adresse de tableau de CASO est E796h. Nous devons donc placer cette valeur dans le registre DE lors de l'appel au sous-programme d'ouverture de fichier (Adresse E6A8h). Nous obtenons donc le listing suivant :

```
10 '[
20 ' ORG $1C00          ;
30 ' LD DE.$E987        ; Adresse de tableau du dispositif CASO.
40 ' LD IY.$02C5        ; Adresse de la partie spécialisée de la RAM.
50 ' LD A.$00           ; Mise à 0 du registre A.
60 ' CALL $E6A8         ; Appel d'ouverture du dispositif.
70 ' RET                ;
80 ']
```

Si vous faites un PRINT#1,"A", le X-07 émettra le son caractéristique d'une sauvegarde de K7, ce qui confirme bien que la sortie K7 a bien été initialisée.

Il nous faut ensuite démarrer le magnétophone car celui-ci n'est pas automatiquement commandé par la routine de sortie de caractères à laquelle nous allons faire appel. Pour ce faire, vous devez inscrire le chiffre 1 dans le bit de poids faible du port de sortie F4h, en faisant OUT(\$F4),1.

Il est maintenant nécessaire de laisser au magnétophone le temps de stabiliser sa vitesse Une petite routine de temporisation conviendra parfaitement à cet usage. Elle utilise la paire de registres BC. Elle sera appelée autant de fois que nécessaire.

De plus, nous ferons précéder la sortie de nos données par trois chiffres "\$00" afin de pouvoir les retrouver à la fin et nous les ferons suivre par trois "\$FF". Lors de la sortie des données, le double registre HL est employé afin de pointer sur les données à sortir.

Voici le "source" de ce programme :

```

10 ' [
20 ' ORG $1C00
30 ' LD DE.$E796
40 ' LD IY.$02C5
50 ' LD A.$00
60 ' CALL $E6A8
65 ' IN A.($F4) ; Le démarrage du magnétophone nécessite la
66 ' SET 0.A ; mise à 1 du bit D0 du port F4 (lignes 65 à
70 ' OUT ($F4).A ; 70).
80 ' CALL #TP ; Temporisation...
90 ' CALL #TP
100 ' LD DE.$02C5
110 ' CALL $E827 ; On assigne la sortie CASO
120 ' LD A.$00 ; Les lignes 120 à 170 signalent le début des
130 ' CALL $E88F ; données par un triple 0
140 ' LD A.$00
150 ' CALL $E88F
160 ' LD A.$00
170 ' CALL $E88F
180 ' LD HL.#DD ; Les lignes 180 à 230 constituent la sortie
190 ' #ST LD A.(HL) ; des données...
200 ' CP $00
210 ' JR Z.#TE
220 ' CALL $E88F
225 ' INC HL
230 ' JR #ST
240 ' #TE LD A.$FF ; Les lignes 240 à 290 signalent la fin des
250 ' CALL $E88F ; données par un triple 255...
260 ' LD A.$FF
270 ' CALL $E88F
280 ' LD A.$FF
290 ' CALL $E88F
291 ' CALL #TP ; Les lignes 291 à 294 constituent l'arrêt du
292 ' IN A.($F4) ; magnétophone...
293 ' RES 0.A
294 ' OUT ($F4).A
300 ' RET
310 ' #TP PUSH BC ; Les lignes 310 à 390 constituent la boucle
320 ' LD B.$00 ; de temporisation.
330 ' #E1 LD C.$FF
340 ' #E2 DEC C
350 ' JR NZ.#E2
360 ' DEC B
370 ' JR NZ.#E1
380 ' POP BC
390 ' RET
400 '#DD DEFB $01,02,03,04,05,06,00 ; données
410 ' ]

```

Voici le listing BASIC de cette routine :

```

10 FOR I=&H1C00 TO &H1C00+103
20 READ A$ : POKE I,VAL("&H"+A$)
30 NEXT I
40 PRINT "EXEC &H1C00 POUR DEMARRER LE PROGRAMME"
50 DATA 11,96,E7,FD,21,C5,2,3E,0,CD,A8,E6,DB,F4,CB,C7
60 DATA D3,F4,CD,54,1C,CD,54,1C,11,CS,2,CD,27,E8,3E,0
70 DATA CD,8F,E8,3E,0,CD,8F,E8,3E,0,CD,8F,E8,21,61,1C
80 DATA 7E,FE,0,28,6,CD,8F,E8,23,18,F5,3E,FF,CD,8F,E8
90 DATA 3E,FF,CD,8F,E8,3E,FF,CD,8F,E8,CD,54,1C,DB,F4
100 DATA CB,87,D3,F4,C9,C5,6,7F,E,FF,D,20,FD,5,20,F8
110 DATA C1,C9,1,2,3,4,5,6,0

```

De plus, on utilise le sous-programme de sortie de caractères situé en E88Fh. Celui-ci est commun à la sortie des données pour tout périphérique.

Si l'on désire travailler avec le magnétophone, le premier dispositif étant la K7, il est impératif de faire appel au sous-programme E827h en plaçant dans le registre DE la valeur 02C5h.

Par contre, pour assigner le deuxième dispositif, on placera dans le registre DE l'adresse 02C5h + 8 (8 octets = 2 octets pour l'adresse de tableau + 6 octets pour les trois registres), c'est à dire 02CDh.

On peut donc initialiser plusieurs dispositifs et passer de l'un à l'autre en assignant chaque fois la sortie au dispositif voulu. On peut ainsi entrer des données sur la prise série du CANON et les sortir sur l'imprimante parallèle ou entrer des données au clavier et les sortir sur l'écran du MINITEL.

Le sous-programme de récupération des données

Nous allons maintenant utiliser CASI pour récupérer les données sauvegardées précédemment. Pour ce faire, il faut d'abord initialiser le dispositif CASI comme dispositif d'entrée.

Ensuite, nous lirons les données parvenant sur la prise K7 jusqu'à trouver le triple \$00. Les données arrivant ensuite sur la prise seront celles recherchées et ce, jusqu'au triple \$FF. Afin de simplifier le programme, nous ne tiendrons pas compte des erreurs d'entrées/sorties. Notons que les personnes désirant quand même traiter ces erreurs le peuvent... En effet, les bits C et Z positionnés à 1 indiquent des problèmes de chargement : à vous de concocter la routine de traitement d'erreurs.

Sur le même principe que précédemment, l'adresse de tableau de CASI est E787h. On fait donc appel au même programme d'initialisation.

En ce qui concerne l'entrée des données, nous ferons appel à une routine spécialisée de la ROM du X-07 située en \$E8D4. Cette routine contient elle-même un appel à la routine "ABORT" permettant de stopper le logiciel sans passer par la touche "ON/BREAK".

La temporisation n'est pas utile ici car les données arrivent de la K7. Par contre, vous devez prêter attention à la durée du traitement entre l'entrée de chaque donnée. Si ce dernier est trop long, vous risquez de rater une ou plusieurs données.

Voici le "source" de ce programme :

```
10 ' [  
20 ' ORG $1C00  
30 ' * INIT#1,"CASI:"          ; Les lignes 30 à 70 initialisent le dispositif 1  
40 ' LD DE.$E787                ; en tant que "CASI" .  
50 ' LD IY.$2C5  
60 ' LD A.$0  
70 ' CALL $E6A8  
80 ' * Démarrage du magnétophone  
90 ' IN A.($F4 )  
100 ' SET 0.A  
110 ' OUT ($F4 ).A  
120 ' * On assigne CASI :  
130 ' LD DE.$2C5  
140 ' CALL $E827  
150 ' * Entrées des données    ; Le triple 0 est recherché des lignes 150 à  
160 ' #E1 CALL $E8D4            ; 210 ...  
170 ' JR NZ.#E1  
180 ' CALL $E8D4  
190 ' JR NZ.#E1  
200 ' CALL $E8D4  
210 ' JR NZ.#E1  
220 ' LD HL.$1B00                ; Les lignes 220 à 280 constituent la  
230 ' #E2 CALL $E8D4            ; recherche de la fin des données ....  
240 ' CP $FF  
250 ' JR Z.#FI  
260 ' LD (HL).A  
270 ' INC HL  
280 ' JR #E2  
290 ' #FI CALL $E8D4            ; Si le code suivant le premier $FF n'est pas  
300 ' CP $FF                    ; $FF , on écrit le code dans la mémoire  
310 ' JR Z.#F1                  ; (lignes 290 à 320) .  
320 ' LD A.(HL)  
330 ' INC HL                    ; On incrémente le pointeur et on continue le  
340 ' JR #E2                    ; chargement ...  
350 ' #F1 CALL $E8D4            ; Si le deuxième code est $FF, on vérifie le  
360 ' CP $FF                    ; troisième ... lignes 350 à 400 .  
370 ' JR Z.#F2  
380 ' LO A.(HL)  
390 ' INC HL  
400 ' JR Z.#E2  
410 ' * Arrêt magnétophone  
420 ' #F2 IN A.($F4)  
430 ' RES 0.A  
440 ' OUT ($F4).A  
450 ' RET  
460 ' ]
```

Voici le listing BASIC de cette routine :

```

10 FOR I=&H1C00 TO &H1C00+82
20 READ A$ : POKE I,VAL("&H"+A$)
30 NEXT I
40 PRINT "EXEC &H1C00 POUR DEMARRER LE PROGRAMME"
50 DATA 11,87,E7,FD,21,CS,02,3E,00,CD,A8,E6,DB,F4,CB,C7
60 DATA D3,F4,11,CS,02,CD,27,E8,CD,D4,E8,20,PB,CD,D4,E8
70 DATA 20,F6,CD,D4,ES,20,F1,21,00,1B,CD,D4,E8,FE,FF,28
80 DATA 04,77,23,18,F5,CD,D4,E8,FE,FF,28,04,7E,23,18,EA
90 DATA CD,D4,E8,8F,FE,FF,28,04,7E,23,18,DF,DB,F4,CB,87
100 DATA D3,F4,C9

```

Les méthodes employées pour tester le début et la fin de l'enregistrement peuvent vous paraître un peu rustiques mais la rapidité prime. On préférera donc faire les tests les uns après les autres plutôt que d'utiliser une boucle. Le calcul de saut pour le microprocesseur risquerait de vous faire perdre des informations.

Les données entrées se retrouvent après l'exécution de cette routine les unes à la suite des autres à partir de l'adresse 1B00h. N'attendez pas une fiabilité exemplaire de ces deux petits programmes... En effet, aucun test ne permet de détecter des erreurs d'entrées/sorties. Le X-07 peut très bien entrer une donnée pour une autre sans que rien ne le prédise.

Nous ne développerons pas ici de routines plus performantes... En effet, celles-ci existent déjà dans la ROM du CANON. Mais rien ne vous empêche désormais d'employer la prise magnétophone pour vous fabriquer, par exemple, un digitaliseur très simple de sons.

Utilisation des fichiers GPR & KBD

Nous allons dans ce paragraphe initialiser l'imprimante graphique comme premier dispositif. Mais afin d'éviter de tomber dans la médiocrité des répétitions intempestives, nous allons initialiser en même temps le clavier comme fichier numéro 2. De cette façon, le X-07 recopiera sur l'imprimante toutes les touches pressées sur son clavier.

Nous avons choisi le clavier comme dispositif d'entrée et non la prise série car cette dernière fait l'objet d'un programme complet. Mais rien ne vous empêche de le faire en tenant compte de la transcription des signaux provenant par exemple d'un MODEM ou d'un MINITEL.

Pour réaliser cette application, il vous faut l'adresse de tableau des dispositifs (GPR --> \$E7B2 et KBD --> \$E778).

Nous placerons lors de l'initialisation dans le registre DE l'adresse de base de la RAM (02C5h) pour le premier dispositif et l'adresse 02CDh pour le deuxième système. De plus, l'initialisation de KBD va vider la mémoire tampon du clavier, ceci nous évitant de l'accomplir...

Les routines d'initialisation des dispositifs ne se distinguent donc pas des précédentes mis à part pour KBD. Notons que l'on peut aussi utiliser le dispositif LPT comme système de sortie (imprimante CENTRONICS). Dans ce but, il suffit de remplacer la valeur E7B2h par E7F8h.

Voici le listing :

```
10 ' [  
20 ' ORG $1C00 ; Les lignes 20 à 70 constituent l'initialisation  
30 ' * INIT #1,"GPR:" ; du dispositif 1...  
40 ' LD DE.$E7B2 ;  
50 ' LD IY.$02C5 ;  
60 ' LD A.$0 ;  
70 ' CALL $F6A8 ;  
80 ' * INIT #2,"KBD:" ; Les lignes 80 à 120 représentent  
90 ' LD DE.$E778 ; l'initialisation du dispositif n°2.  
100 ' LD IY.$02CD ;  
110 ' LD A.$0 ;  
120 ' CALL $E6A8 ;  
130 ' * Assignment de KBD ; Entrées sur la console...  
140 ' #BL LD DE.$2CD ;  
150 ' CALL $E827 ;  
170 ' CALL $E8D4 ; Entrée d'une touche...  
175 ' PUSH AF ;  
190 ' LD DE.$2C5 ; Les sorties se font sur l'imprimante  
200 ' CALL $E827 ;  
205 ' POP AF ;  
206 ' CP $0E ; Si on appuie sur CTRL N , on arrête  
207 ' RETZ ;  
210 ' CALL $E88F ; Sinon, écriture sur l'imprimante.  
220 ' JR #BL ;  
230 ' ]
```

Voici le listing BASIC de cette routine :

```
10 FOR I=&H1C00 TO &H1C00+48  
20 READ A$: POKE I,VAL("&H"+A$): NEXT  
40 PRINT "EXEC &H1C00 pour démarrer... "  
50 DATA 11,F8,E7,FD,21,CS,02,3E,00,CD,A8,E6,11,78,E7  
60 DATA FD,21,CD,02,3E,00,CD,A8,E6,11,CD,02,CD,27,E8,CD,D4  
70 DATA E8,F5,11,CS,02,CD,27,E8,F1,FE,0E,C8,CD,8F,E8,18,E7
```

Ce programme pourrait être plus rapide en utilisant directement le port F1h sur lequel se retrouvent les touches pressées. Mais le but de cette routine étant d'employer les routines d'entrées/sorties, ceci se révélerait inutile.

De plus, cela nous conduirait à tester si les données se trouvent bien sur le port F1h alors que la routine \$E8D4 gère automatiquement cela. En effet, elle ne rend la main que si une donnée est présente.

Utilisation de la prise série

Avec le contenu de ce paragraphe, vos routines vont prendre le Concorde pour Rio de Janeiro !!! En effet , en langage machine , les possibilités de traitement deviennent grandioses car le temps séparant l'entrée et la sortie de deux mots peut être considérablement réduit.

Nous allons nous limiter à deux programmes, l'un servant à écrire sur le MINITEL et l'autre servant à y lire.

Lors de l'initialisation du dispositif, il est important d'indiquer la vitesse de transmission et le mode ACIA. Par conséquent, nous allons employer deux registres du NSC800... La vitesse de transmission sera chargée dans le registre IX et le mode ACIA dans le registre B. De plus, l'adresse de tableau du dispositif COM est \$E7A4.

Le premier programme inscrira à l'écran du MINITEL un message après avoir effacé l'afficheur. Dans cette optique, on pointe le registre HL sur le message. Après l'affichage de chaque lettre, si le caractère suivant est un 0, le message sera terminé et la routine s'arrêtera.

Pour effacer l'écran, il suffit d'envoyer le code 12 qui correspond à la commande CLS du MINITEL. On utilise toujours le même dispositif d'initialisation commun à tous les systèmes.

Voici le listing de cette routine :

```
10 ' [
20 ' ORG $1C00      ;
30 ' LD DE.$E7A4    ; Dispositif COM activé
40 ' LD A.$0        ;
50 ' LD B."G"       ; Mode G de l'ACIA
60 ' LD IX.&1200     ; Vitesse : 1200 bauds
70 ' LD IY.$2C5     ;
80 ' CALL $E6A8     ; Ouverture...
85 ' LD DE.$2C5     ;
86 ' CALL $E827     ;
87 ' LD HL.#ME      ; Ecriture du message...
90 ' #ET LD A.(HL)  ;
91 ' CP $0          ;
92 ' JR Z.#FI       ;
100 ' CALL $E88F    ;
105 ' INC HL        ;
106 ' JR #ET        ;
120 ' #FI RET       ;
124 ' #ME DEFB &12  ;
125 ' DEFM Cela marche bien comme cela...
126 ' DEFB $00      ;
130 ' ]
```

Voici le listing BASIC :

```

10 FOR I=&H1C00 TO &H1C00+63
20 READ A$: POKE I,VAL("&H"+A$): NEXT
40 PRINT "EXEC &H1C00 pour démarrer... "
60 DATA 11,A4,E7,3E,00,06,47,DD,21,B0,04,FD,21,CS,02
70 DATA CD,A8,E6,11,C5,02,CD,27,E8,21,27,1C,7E,FE,00,28,06
80 DATA CD,8F,E8,23,18,F5,C9,0C,43,41,20,4D,41,52,43,48
90 DATA 45,20,42,49,45,4E,20,43,4F,4D,4D,45,20,43,41,00

```

Occupons-nous pour finir ce paragraphe de la lecture des données provenant d'un périphérique sériel. Cela occasionne peu de changements. En effet, l'initialisation demeure identique. Le changement réside dans l'appel de la routine de lecture au lieu de celle de l'écriture, la donnée se trouvant alors dans le registre A. Un appel à l'adresse \$C1BE fait d'ailleurs apparaître cette donnée.

Voici le listing de cette routine :

```

20 ' [
30 ' ORG $1C00      ;
40 ' LD DE.$E7A4    ; Les lignes 40 à 90 initialisent le 1er fichier
50 ' LD A.$0        ; à COM.
60 ' LD B."G"       ;
70 ' LD IX.&1200     ;
80 ' LD IY.$2C5     ;
90 ' CALL $E6A8     ;
100 ' LD DE.$2C5    ; Les lignes 100 à 110 assignent l'entrée.
110 ' CALL $E827    ;
120 ' CALL $E8D4    ; Caractère entré dans A.
130 ' CALL $C1BE    ; Caractère affiché à l'écran
140 ' RET           ;
150 ' ]

```

Voici le listing BASIC :

```

20 FOR I=&H1C00 TO &H1C00+29
30 READ A$: POKE I,VAL("&H"+A$): NEXT
50 PRINT "EXEC &H1C00 pour démarrer ... "
60 DATA 11,A4,E7,3E,00,06,47,DD,21,B0,04,FD,21,C5,02,CD
70 DATA A8,E6,11,C5,02,CD,27,E8,CD,D4,E8,CD,DE,C1,C9

```

Conclusion

Pour conclure ce chapitre, nous vous présentons le "DUMP" de la table mémoire où se trouvent placées l'adresse des sous-programmes des dispositifs et la liste d'adresses de tableau de chaque système.

Dispositif	Adresse en hexa	Commentaires
RAM	E7CE	
CON	E7EA	
KBD	E778	
GPR	E7B2	
LPT	E7F8	
CASI	E787	
CASO	E796	
OPT	E7DC	
COM	E7A4	
PRT	E7C0	

Après toutes ces émotions, nous allons pouvoir attaquer la X-720... Nous espérons que ce chapitre très important concernant les interruptions et les entrées/sorties vous aura apporté toutes les solutions dont vous rêviez depuis si longtemps !!

Pour le moment, cap sur la vidéo "made in CANON"...

L'interface X-720

Ce sixième chapitre est entièrement consacré à un périphérique relativement controversé la X-720 ou, en termes simples, l'interface péritélévision. Comme son nom l'indique, elle permet de relier le X-07 à un poste équipé de la prise péritel.

Assez encombrante et non autonome, elle ajoute un regrettable "fil à la patte" au X-07 qui perd alors sa vocation de portable pour se consacrer au familial. Malheureusement, le peu de logiciels, sur le marché, adaptés à cette interface n'ont pas révélé les intéressantes possibilités dont elle est pourtant dotée.

Nous allons à travers deux approches opposées (le HARD et le SOFT) vous exposer quelques mystères de la X-720.

L'approche « HARD »

Généralités

La X-720 est architecturée autour d'un générateur d'affichage vidéo de la célèbre firme MOTOROLA, le MC6847. Mais ce processeur n'est pas seul, de la RAM et de la ROM se partagent ses faveurs.

La mémoire vive de six kilo-octets représente la capacité maximale du MC6847. Elle est présente sous la forme de trois circuits mémoire de deux kilo-octets chacun.

En ce qui concerne la mémoire morte, deux boîtiers se partagent les fonctions. L'un contient les caractères affichables à l'écran et l'autre un interpréteur BASIC donnant accès à de nouvelles fonctions.

Une étude du diagramme de la X-720 met clairement en évidence la manière dont sont stockées les données présentes à l'affichage. Deux des circuits de la mémoire vive contiennent l'état d'un point tandis que le dernier circuit contient sa couleur.

FIGURE 8 : DIAGRAMME DE LA X-720

Ad.	Uti.	Lecture								Ecriture							
		D7	D6	D5	D4	D3	D2	D1	D0	D7	D6	D5	D4	D3	D2	D1	D0
80-8F	C.G. ROM	Lecture des informations								Adresse du C.G. ROM							
90-97	----	FS	INT			B4	B3	B2	B1	FSR	A/G	+1K	GM2	GM1	GM0	CSS1	CSS0
98-9F	E/S									Adresse basse d'une cartouche d'entrée/sortie							

FIGURE 9 : TABLE DES E/S

La communication

La communication entre le X-07 et la X-720 s'effectue via les ports d'entrée/sortie pour les échanges avec le MC6847. Par contre, les bus d'adresses et de données se chargent des communications avec la mémoire vive (VRAM).

Le plan détaillé de l'utilisation des ports d'entrée/sortie employés pour communiquer avec le X-07 est représenté par la figure numéro 9.

Voici les détails :

Adresses 80 à 8F : le processeur utilise ces adresses pour lire la CGROM. Précisons que la CGROM constitue la ROM contenant le générateur de caractères. En fait, il s'agit d'une mémoire morte stockant les caractères sous forme binaire, le chiffre 1 signifie alors un point allumé et le chiffre 0 un point éteint. Nous pouvons remarquer que cette méthode est en tout point identique à la méthode de programmation des caractères redéfinissables.

Adresses 90 à 97 (en lecture) : les bits B1 à B4 ne sont pas utilisés. Par contre le bit FS mis à l'état bas permet au processeur d'effectuer un scrolling de l'écran. De plus, le bit INT constitue un bit d'interruption utilisé pour l'affichage du curseur.

Adresses 90 à 97 (en écriture) : le bit FSR interdit ou autorise les interruptions d'affichage du curseur (lorsque ce bit est à 0, le curseur n'est pas affiché). Le bit A/G est un drapeau de mode graphique. Le bit +1K modifie l'adresse de la VRAM (Ex : modification de l'adresse de la page vidéo affichée). Les bits GM0 à GM2 constituent les drapeaux de sélection des modes graphiques. Enfin, les bits GSS0 et GSS1 sont des drapeaux divers.

Répartition de la mémoire vidéo

La répartition de la VRAM se fait en fonction des différents modes d'écran. A cause des divers modes de résolution et de la présence de plusieurs pages d'écran, l'adressage de la mémoire vidéo diffère d'un cas sur l'autre.

En fait, pour chaque affichage, une partie d'un des trois circuits est utilisée. Les schémas présentés un peu plus loin vont vous permettre de reconnaître la mémoire occupée par chaque type de définition, les zones hachurées représentent la mémoire utilisée.

FIGURE 10 : MEMOIRE OCCUPEE

En fait, à la lecture de ces schémas, il s'avère que l'adresse rencontrée par le processeur ne varie pas d'une page graphique à l'autre. En effet, le signal +1K (adresses 90 à 97 des entrées/sorties) est employé dans le but de modifier l'espace mémoire adressé. Les trois circuits de deux kilo-octets sont divisés en six segments d'un kilo-octet chacun. Par conséquent, le passage d'une partie du circuit à une autre se réalise grâce au signal +1K provoquant ainsi le changement immédiat de l'image affichée à l'écran. On devine la complexité de la gestion d'un tel système. En effet, pour afficher un point ou une lettre à l'écran, l'interpréteur doit calculer l'adresse des valeurs à modifier en fonction du mode et de la page écran ceci explique la grande lenteur de l'interface X-720.

De plus cette réelle complexité rend très ardue l'utilisation des nombreuses routines qui existent dans la mémoire morte du X-07. La programmation en langage machine de la vidéo, en faisant appel à ces routines n'offre donc que peu d'intérêt, étant donné la lenteur d'exécution de celles-ci. Il est préférable de se placer dans un mode donné et de créer des fonctions SET et RESET pour ce mode particulier. Vous pouvez utilement vous référer au programme "Le piège - version vidéo" présent au dernier chapitre il a été écrit en tenant compte de ces remarques.

Avant de passer au SOFT proprement dit, les électroniciens pourront admirer le plan détaillé du circuit de cette interface... Bon courage !

L'approche « SOFT »

L'interface X-720 possède donc six kilo-octets de mémoire vive (mémoire écran) implantée entre 8000h et 97F F h et quatre kilo-octets de mémoire morte implantée entre A000h et AFFFh (complément à l'interpréteur du CANON). Cette RAM débute par le mot-clé "love", recherché à chaque mise sous tension par l'interpréteur.

Derrière ce mot-clé bien connu de tous les canonistes, l'adresse A0CAh du sous-programme d'initialisation de la zone de communication (nouvelles fonctions : PAINT, COLOR, SCREEN et modification de la plage d'action des commandes PSET, PRESET, POINT, LINE, CIRCLE) est présente.

Sans plus attendre , nous allons vous disséquer les points logiciels intéressants de cette interface.

Les points d'entrée

Voici les points d'entrée des fonctions BASIC utilisables avec la X-720.

Quelques adresses

Le sous-programme d 'initialisation plante dans la zone système du X-07 quelques valeurs que nous allons vous exposer.

Fonctions utilisables

Un désassemblage plus profond de la ROM de la X-720 a permis de mettre en lumière quelques fonctions utilisables dont voici les adresses.

De plus, la résolution étant différente pour chaque mode d'écran, les ingénieurs ont été obligés d'écrire des routines graphiques correspondant à ces définitions. Le problème a été résolu en plaçant entre A9FB h et AA73h, une table contenant l'adresse de chacun des sous-programmes d'exécution des fonctions, en fonction du mode choisi. Ces sous-programmes se trouvent d'ailleurs stockés de A631h à A9FAh.

Codage des pixels en fonction des différents modes

Comme nous l'avons déjà souligné, les six modes possibles rendent le fonctionnement de l'interpréteur extrêmement complexe et ceci implique une grande lenteur d'exécution.

En effet, pour chaque fonction graphique, l'interpréteur doit décoder ou calculer les paramètres puis, en fonction du mode écran et de la page active il doit déterminer l'adresse du pixel à modifier.

Examinons de plus près le codage de ces pixels.

Mode 1 : le texte seul est affiché. On a accès à 16 lignes de 32 colonnes soit 512 octets. C'est le générateur de caractères de la X-720 qui permet d'afficher à l'écran le contenu (valeur ASCII) de chaque case mémoire.

Mode 2 : du texte ou du graphisme peuvent être affichés. Le générateur de caractères intervient quand du texte est affiché. Par contre, quand du graphisme est reproduit à l'écran, il faut calculer les coordonnées du pixel (six pixels par case mémoire + deux bits pour la couleur du pixel). Vous pouvez utilement vous reporter à la figure exposée à la page suivante.

Modes 3 et 5 : ces deux modes sont graphiques. Les caractères sont dessinés pixel par pixel. Notons que les pixels sont adressés quatre par quatre et qu'un octet contient quatre pixels (donc, quatre couleurs possibles). La figure présentée à la page suivante explicite ce codage

Modes 4 et 6 : ces deux modes sont graphiques. Les remarques sont les mêmes en ce qui concerne les caractères. A chaque pixel correspond un bit de la mémoire écran, les pixels sont donc adressés huit par huit. En mode 4, trois kilo-octets de la mémoire écran sont utilisés. D'autre part, 512 octets des trois kilos restants sont utilisables en mode 2. Enfin, en mode 6, les six kilo-octets sont utilisés; en effet, la définition est doublée.

Nous allons pouvoir utiliser nos connaissances toutes neuves de la X-720 dans de petites routines qui vont bouleverser votre écran.

Utilitaires

X-720 présente ?

La première routine teste la présence de la X-720 dans le port d'extension de votre CANON.

Le principe est très simple, la routine va écrire une valeur déterminée en haut de la VRAM. En relisant l'octet, si la valeur est différente, un traitement d'erreur sera effectué car la X-720 n'aura pas été connectée.

La valeur 80h est stockée à l'adresse 8000h. En relisant cette adresse, la X-720 sera connectée si la valeur lue est 80h. Simple, non ?

Effacement partiel de l'écran

Ce petit programme permet d'effacer les huit dernières lignes de l'écran. La position du curseur est stockée aux adresses B8h et B 9h comme pour le curseur de l'écran LCD le

principe est de simuler une coupure de l'écran et d'effacer tout ce qui se trouve en dessous de cette coupure par la routine CE9Eh.

La fonction SCREEN

Pour simuler cette fonction, la routine AB09h de la ROM a été mise à contribution. En effet, la page active, 1 est stockée dans le registre D et la page visuelle, 1 est contenue dans le registre E.

Le mode écran a été placé dans le registre C et à l'adresse 00D1h.

La fonction COLOR

Deux méthodes sont possibles pour résoudre ce problème. La première solution (la plus simple) consiste à charger le registre A avec le numéro de la couleur désirée puis d'appeler la routine COLOR par un CALL A637h.

La deuxième méthode, reprise par la routine exposée, permet de changer la couleur du fond, des caractères et de la palette utilisée. Qui a dit que nous recherchions la complication ?

Les listings sont présents dans les pages suivantes. Ensuite, place aux applications.

```

10 ' [
20 ' ORG $1C00
30 ' * TEST DE LA PRESENCE DE LA X-720
40 ' LD A,$80
50 ' LD ($8000).A : * DEBUT VRAM
60 ' LD A.($8000)
70 ' CP $80
80 ' JP NZ.$F1AA: * SAUT EN ERREUR
90 ' RET
100 ' ]

```

```

1C00 3E 80      LD A,80
1C02 32 00 80   LD ($8000),A
1C05 3A 00 80   LD A,($8000)
1C08 FE 80      CP $80
1C0A C2 AA F1   JP NZ,$F1AA
1C0D C9         RET

```

```

10 REM TEST DE LA X-720
20 FOR I=&H1C00 TO &H1C0D
30 READ A$: POKE I,VAL("&H"+A$)
40 NEXT I
50 DATA 3E,80,32,00,80,3A,00,80,FE,80
60 DATA C2,AA,F1,C9

```

FIGURE 12 : TEST

```

10 ' [
20 ' ORG $1C00
30 ' * EFFACEMENT PARTIEL DE L' ECRAN
40 ' CALL #SC : * EFFACEMENT DE LA
50 ' CALL $CE9E : * 8ème LIGNE à LA FIN
60 ' LD HL.$108
70 ' LD ($B8).HL : * CURSEUR (C1, L8)
80 ' RET
90 ' #SC LD A.$8 : * Ligne de roulement
100 ' LD ($BB).A
110 ' LD A.&16 : * DERNIERE LIGNE
120 ' LD ($BC).A
130 ' LD A.(&16-8+1) : * Nb de lignes
140 ' LD ($BD).A
150 ' RET
160 ' ]

```

```

1C00 CD 0D 1C      CALL 1C0D
1C03 CD 9E CE      CALL CE9E
1C06 21 08 01      LD HL,$0108
1C09 22 B8 00      LD ($00B8),HL
1C0C C9            RET
1C0D 3E 08         LD A,$08
1C0F 32 BB 00      LD ($00BB),A
1C12 3E 10         LD A,$10
1C14 32 BC 00      LD ($00BC),A
1C17 3A 10 00      LD A,($0010)
1C1A 32 BD 00      LD ($00BD),A
1C1D C9            RET

```

```

10 REM EFFACEMENT PARTIEL
20 FOR I=&H1C00 TO &H1C1D
30 READ A$: POKE I,VAL("&H"+A$)
40 NEXT I
50 DATA
CD,0D,1C,CD,9E,CE,21,08,01,22,B8,00,C9
60 DATA
3E,08,32,BB,00,3E,10,32,BC,00,3A,10,00
70 DATA 32,BD,00,C9

```

FIGURE 13 : EFFACEMENT

```

10 ' [
20 ' ORG $1C00
30 ' *** SCREEN ***
40 ' LD A.$00 : * SCREEN 1
50 ' LD ($D1).A
60 ' LD C,A
70 ' LD A.$01
80 ' LD D.A : * PAGE ACTIVE - 1
90 ' LD E.A : * PAGE VISUELLE - 1
100 ' JP $AB09
110 ' ]

```

```

1C00 3E 00          LD A,$00
1C02 32 D1 00       LD ($00D1),A
1C05 4F             LD C,A
1C06 32 01          LD A,$01
1C08 57             LD D,A
1C09 5F             LD E,A
1C0A C3 09 AB       JP $AB09

```

```

10 REM *** SCREEN ***
20 FOR I=&H1C00 TO &H1C0C
30 READ A$: POKE I,VAL("&H"+A$)
40 NEXT I
50 DATA 3E,00,32,D1,00,4F,3E,01,57
60 DATA 5F,C3,09,AB

```

FIGURE 14 : SCREEN

```

10 ' [
20 ' ORG $1C00
30 ' *** COULEUR ***
40 ' LD A.$01
50 ' LD ($4E5).A : * COULEUR LETTRES
60 ' LD A.$02
70 ' LD ($4E6).A
80 ' LD A.$00
90 ' LD ($4E7).A : * PALETTE 0 ou 2
100 ' RET
110 ' ]

```

```

1C00 3E 01          LD A,$01
1C02 32 E5 04       LD ($04E5),A
1C05 3E 02          LD A,$02
1C07 32 E6 04       LD ($04E6),A
1C0A 3E 00          LD A,$00
1C0C 32 E7 04       LD ($04E7),A
1C0F C9             RET

```

```

10 REM *** COLOR ***
20 FOR I=&H1C00 TO &H1C0F
30 READ A$: POKE I,VAL("&H"+A$)
40 NEXT I
50 DATA 3E,01,32,E5,04,3E,02,32,E6,04
60 DATA 3E,00,32,E7,04,C9

```

FIGURE 15 : COLOR

Embarquons-nous...

Cap : « les applications »

3^{ème} partie

... Et où l'on termine « LOGICIEL » !

Introduction

Enfin. Nous voici arrivés à la dernière partie de cet ouvrage. Et quelle partie.

En effet, vous allez pouvoir vous régaler en savourant les délicieux logiciels que nous vous avons soigneusement préparés. Des utilitaires aux jeux, en passant par la poésie, le menu est particulièrement appétissant.

Mais sans plus attendre, voici quelques indications avant de vous précipiter corps et âme sur les pages suivantes.

Chaque logiciel est expliqué en détail : spécifications, principe, utilisation, résultats et listing sont présentés le plus clairement possible.

D'autre part, deux types de programmes sont exposés : ceux écrits exclusivement en BASIC et ceux composés en ASSEMBLEUR.

Pour ceux qui ne possèdent pas la K7 (Ils sont très courageux), nous vous conseillons de relire plusieurs fois les codes entrés car la moindre erreur peut être fatale. Un "entrepreneur de codes" leur servira à entrer les codes en mémoire et un "chargeur" leur permettra de sauvegarder et de récupérer le logiciel via une K7.

Evidemment, les personnes possédant la K7 n'ont pratiquement rien à faire. L'ordre CLOAD est à utiliser avec les logiciels écrits uniquement en BASIC et l'ordre CLOAD puis RUN pour les logiciels ASSEMBLEUR. En effet, ces derniers possèdent un chargeur BASIC qui permet de stocker les codes rapidement en mémoire. D'ailleurs un RUN 100 permet de sauvegarder ces codes sur K7 afin de faire des copies protectrices.

Nous conseillons à toutes les personnes ne possédant pas la K7 de se la procurer au plus vite auprès des éditions NEPTUNE. En effet, en dehors de toute considération commerciale, il vaut mieux acquérir cette K7 car certains logiciels écrits en ASSEMBLEUR sont particulièrement longs.

Voilà. Nous espérons que vous apprécierez les divers logiciels présents dans cette dernière partie ils sont diversifiés et utilisent pleinement les notions abordées dans le présent ouvrage. Analysez-les avec soin.

LMDATA & Applications

La transcription en DATA d'une routine écrite en ASSEMBLEUR constitue une opération fastidieuse et génératrice d'erreurs ! Pourquoi ne pas confier ce travail à l'utilitaire "LMDATA" ?

L'utilisation de LMDATA est la suivante :

- Implanter la routine L.M. (à transcrire en DATA) en mémoire.
- Lancer LMDATA après l'avoir chargé dans le X-07.
- Supprimer les lignes de DATA superflues puis les lignes 1 à 230. Cette opération peut se faire à l'aide de la carte XP-140 (commande DEL), avec le fichier 5, l'éditeur BASIC, etc.
- Si la routine LM accuse plus de 256 octets, il faut rajouter des lignes de DATA à la suite du logiciel initial. De plus, par mesure de prudence, ne pas exécuter LMDATA sans avoir effectué une copie K7 ou fichier RAM de la routine à implanter en DATA.

Le principe de LMDATA est le suivant :

Ligne 40 : calcul de l'adresse absolue du premier octet de la ligne 10000.

Ligne 50 : entrée boucle des lignes complètes.

Ligne 60 : écriture de la ligne, calcul de l'adresse de la ligne suivante.

Ligne 70 : nombre de données des lignes incomplètes.

Ligne 200 : boucle des lignes de DATA.

Ligne 210 : calcul des variables C, D et U.

Ligne 220 : écriture des valeurs ASCII des variables C, D, U dans le programme puis ajustement de l'adresse mémoire.

Trois exemples sont joints à cet utilitaire : le RESTORE calculé que nous avons déjà vu en début d'ouvrage, une copie rapide de l'écran et un graphisme sur écran LCD représentant le LOGO CANON.

Longueur de LMDATA : 1670 octets.

Longueur du LOGO CANON : 831 octets.

Longueur du RESTORE calculé : 372 octets.

Longueur de COPIE RAPIDE : 539 octets.

Implantation de ces logiciels : BASIC.

```

1 REM Ecriture automatique de codes LM
en DATA
10 CLEAR 50,&H1F00: DEFINT A-Z: LL=16:
CA=48: CLS
20 INPUT"Adres. debut routine";DM: DD=DM
30 INPUT"Adresse fin routine";FM: NC=FM-
DM+1: JC=INT(NC/LL)
40 RESTORE 10000:
AD=PEEK(&H328)+256*PEEK(&H329)+6
50 LB=LL: FOR J=1 TO JC
55 PRINT"LIGNE";10000+10*J
60 GOSUB 200: AD=AD+5: NEXT J
70 LB=NC-JC*LL: IF LB=0 THEN 100
80 GOSUB 200
90 PRINT"Effacer la fin de la
ligne";10000+10*JC
100 PRINT"Effacer les
lignes";10000+10*(JC+1);"";10150;
110 IF INKEY$="" THEN 110
120 END
200 FOR I=1 TO LB: VC=PEEK(DM): DM=DM+1
210 C=INT(VC*.01): D=INT((VC-100*C)/10):
U=VC-10*D-100*C
220 POKE AD,C+CA: POKE AD+1,D+CA: POKE
AD+2,U+CA: AD=AD+4
230 NEXT: RETURN
10000 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10010 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10020 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10030 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10040 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10050 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10060 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10070 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10080 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10090 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10100 DATA ...,...,...,...,...,...,
...,...,...,...,...,...

```

FIGURE 16 : LMDATA

```

10110 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10120 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10130 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10140 DATA ...,...,...,...,...,...,
...,...,...,...,...,...,
10150 DATA ...,...,...,...,...,...,
...,...,...,...,...,...

```

```

1 REM Exemple de RESTORE CALCULE.
2 REM Cree par LMDATA
10 CLEAR 50,&H1FFF: DEFINT A-Z
20 RESTORE 10000
30 FOR I=0 TO 17: READ A: POKE
&H2000+I,A: NEXT
40 INPUT"N= (1..3)";N
50 IFN<1 OR N>3 THEN 40
60 J=N*100
70 Z=USR(&H2000,J)
80 READ A$: PRINT A$: GOTO 40
100 DATA"lecture DATA 100"
200 DATA"lecture DATA 200"
300 DATA"lecture DATA 300"
999 END
10000 DATA 035,035,094,035,086,205,013,
243,096,105,210,056,246,043,034,040
10010 DATA 003,201

```

FIGURE 16 : FIN

FIGURE 17 : EXEMPLE

```

5 REM EXEMPLE DE GRAPHISME

10 CLEAR 200: DEFINT A-Z
20 X=23: CLS
30 FOR L=6 TO 25
40 A1=1: F=1
50 READ A: A1=A1+A: IF F=1 THEN 70
60 F=1: GOTO 80
70 FOR I=A1-A TO A1-1: PSET(X+I,L):
NEXT: F=0
80 IF A1<79 THEN 50
90 NEXT
100 END
110 DATA 79,9,4,66,8,6,65,6,10,63
120 DATA 5,12,7,4,9,3,3,2,7,7,6,3,3,2,6
130 DATA 4,6,6,2,5,6,7,5,1,4,5,9,4,5,1,
4,5
140 DATA 4,5,6,4,2,9,4,13,3,2,3,6,1,13,4
150 DATA 3,6,6,2,10,3,6,5,2,5,2,3,2,6,3
,5,2,5,3
160 DATA 3,5,6,1,12,4,5,5,2,5,1,4,3,6,2
,5,2,5,3
170 DATA 3,5,15,8,5,5,2,5,1,4,3,6,2,5,2
,5,3
180 DATA 3,5,14,10,4,5,2,5,1,5,3,5,2,5,
2,5,3
190 DATA 3,5,13,5,2,4,4,5,2,5,1,5,3,5,
2,5,2,5,3
200 DATA 3,6,11,5,4,4,3,5,2,5,1,6,3,4,
2,5,2,5,3
210 DATA 4,5,9,1,1,5,4,4,3,5,2,5,1,6,3,
4,2,5,2,5,3
220 DATA 4,6,6,2,2,6,2,6,2,5,2,5,2,6,2,
3,3,5,2,5,3
230 DATA 5,12,4,8,1,4,2,5,2,5,2,6,3,2,
3,5,2,5,3
240 DATA 6,10,6,6,2,5,1,5,2,5,3,9,4,5,
2,5,3
250 DATA 8,6,9,4,3,5,1,5,2,5,4,7,5,5,2
,5,3,79,79

```

FIGURE 18 : LOGO CANON

```

1 REM Copie écran
2 REM Méthode de la chaîne
3 REM DATA créées par LMDATA.
10 CLEAR 100: DEFINT A-Z
20 CH$=STRING$(53,"F")
30 GOSUB 300
40 FOR I=0 TO 52: READ A: POKE AD+I,A:
NEXT
100 PRINT" Copie rapide de l'"
110 PRINT"      écran sur"
120 PRINT" l'imprimante X-710"
130 PRINT"      +++";
200 GOSUB 300:EXEC AD
210 END
300 AD=VARPTR(CH$):
AD=PEEK(AD+1)+256*PEEK(AD+2): RETURN
10000 DATA 006,000,033,020,002,197,229
,001,019,000,009,126,254,032,032,004
10010 DATA 043,013,032,247,012,225,229
,126,004,035,229,197,205,247,206,193
10020 DATA 225,120,185,032,242,205,176
,207,225,001,020,000,009,193,004,120
10030 DATA 254,004,032,209,201

```

FIGURE 19 : COPIE RAPIDE

LOGOGENESE

" Poètes, artistes, littérateurs, écrivains, admirez ce pur produit de la science informatique... Adorez LOGOGENESE, le premier logiciel informatique qui va remplacer les académiciens !!"

Non, non , n'ayez pas peur... Nous sommes en train de délirer mais cela va passer !! En restant sérieux , LOGOGENESE est un programme de création de mots : ils se forment au hasard à partir de syllabes stockées en DATA.

Evidemment , les mots ainsi créés ne vont pas avoir beaucoup de sens mais que voulez-vous, il faut bien dévoiler son originalité de temps en temps...

Le programme est très simple en lui-même. Ecrit entièrement en BASIC, il fait appel aux méthodes du RESTORE calculé et de la chaîne de caractères pour l'implantation d'une petite routine.

Pour l'utiliser, faire RUN suivi de RETURN dès que le programme se trouve en mémoire... A partir de ce moment-là, le X-07 crée des mots et les affiche à l'écran dans un bouillonnement de culture !!

Longueur de LOGOGENESE: 1819 octets.

Implantation de LOGOGENESE: BASIC.

```

1 'ATTENTION NE PAS RENUMEROTER
10 DEFINT A-Z: CLS: PRINT"Logogénese"
20 RM$=STRING$(18,0): GOSUB 90
30 FOR I=0 TO 17: READ B$: POKE AD+I,
VAL("&H"+B$): NEXT: Z=RND(0)
40 DATA 23,23,5E,23,56,CD,0D,F3,60,69,
D2,38,F6,2B,22,28,03,C9
45 J=INT(RND(1)*33)+100: GOSUB 90
50 Z=USR(AD,J): N=INT(RND(1)*4+1)
55 FOR I=1 TO N: READ D$: NEXT
60 K=INT(RND(1)*14)+200: GOSUB 90
65 Z=USR(AD,K): N=INT(RND(1)*4+1)
70 FOR I=1 TO N: READ F$: NEXT
75 PRINT D$;F$: IF INKEY$="" THEN 45
80 IF INKEY$="" THEN 80 ELSE 45
90 AD=VARPTR(RM$):
AD=PEEK(AD+1)+256*PEEK(AD+2): RETURN
95 END

```

FIGURE 20 : LOGOGENESE

```

100 DATA AERO,AGRO,AMBI,ANDRO
101 DATA ANTI,ASTRO,BIBLIO,BIO
102 DATA CACO,CALLI,CLEPTO,CHIMIO
103 DATA CHRONO,CINE,COPRO,COSMO
104 DATA CRYO,CRYPTO,DECI,DECA
105 DATA HEMI,DERMO,DEXTRO,DODECA
106 DATA DROMO,DYNAMO,DYS,ECTO
107 DATA ELECTRO,EMBRYO,ENTOMO
108 DATA ERGO,EROTICO,GASTRO,GEO
109 DATA GLOSSO,GONIO,GONO,GYMNO
110 DATA HECTO,HELIO,HEMATO,HETERO
111 DATA HIERO,HIPPO,HOLO,HOMEO
112 DATA HOMO,HYDRO,HYPER,HYPO
113 DATA ICONO,IDEO,INFRA,ISO
114 DATA INTRA,LATERO,LIPO,LOGO
115 DATA LOXO,MACRO,MEGALO,METEO
116 DATA META,MICRO,MACRO,MNEMO
117 DATA MORPHO,MYTHO,NECRO,NEO
118 DATA NOSO,NUGLEO,OCTO,OLEO
119 DATA OMNI,ORTHO,PALEO,PAN
120 DATA PARA,PAPYRO,PATHO,PEDO
121 DATA PENTA,PERI,PETRO,PHAGO
122 DATA PHALLO,PHANERO,PHILO,PHYTO
123 DATA PHOBO,PHONO,PHOTO,PHRENO
124 DATA PHYSIO,POLY,MONO,HEMI
125 DATA PORNO,POST,PROTO,PSEUDO
126 DATA PSYCHO,RADIO,RETRO,RHINO
127 DATA SADO,STEREO,SCLERO,SEMI
128 DATA SCHIZO,SIMILI,SPELEO,STENO
129 DATA SUB,SUPRA,SUPER,TACHY
130 DATA TECHNO,TELE,TELEO,THEO
131 DATA THERMO,TRIBO,ULTRA,VIDEO
132 DATA XENO,XYLO,ZOO,MINI
200 DATA ALGIE,CARDE,CINESE,CEPHALE
201 DATA CLASTE,COSMOS,CYCLE,DACTYLE
202 DATA DERME,DIDACTE,DOXE,DROME
203 DATA DYNE,FUGE,GAME,GASTRE
204 DATA GLOTTE,GENESE,GENE,GRADE
205 DATA GYNE,LATERE,LATRE,LOGIE
206 DATA LOGUE,MANCIE,MANE,MANIE
207 DATA MATIQUE,MEGALIE,METRIE,MNESIE
208 DATA MORPHE,NAUTE,NEVROSE,NOMIE
209 DATA PATHE,PHAGE,PHANIE,PHILE
210 DATA PHOBE,PHONE,PNEE,PYGE
211 DATA RRHEE,SCAPHE,SCOPE,SCOPIE
212 DATA STASE,STENIE,THEISME,TONIQUE
213 DATA TROPE,TYPE,VORE,TIQUE

```

FIGURE20 : FIN

LLIST

Cet utilitaire est destiné à améliorer la présentation des listings BASIC : titre, numérotation des pages, saut de ligne en fin de page, alignements des débuts de ligne, formatage des numéros de ligne.

Le mode d'emploi de LLIST est le suivant :

- Vérifier au moyen des commandes DIR et FSET qu'il reste au moins 600 octets libres dans la zone des fichiers RAM. En effet, la partie active du logiciel va s'implanter automatiquement dans le fond de la mémoire.
- Faire CLOAD puis RUN pour charger les codes si vous possédez la K7 des Editions NEPTUNE. Sinon, bon courage !!
- Ajuster la longueur des pages (octet &H950) et la longueur des lignes (octet &H93C) à votre convenance. Pour ajuster ces octets, on peut utiliser la carte XP-140 ou la commande POKE. Remarquons que l'octet &H93C contient la valeur &H27 : il est préférable de la remplacer par &H26 pour éviter des sauts de ligne imprévus si une ligne de 39 caractères survient...
- Lancer le programme par EXEC &H800. La routine retourne alors à l'adresse décimale de son implantation : cette adresse est variable suivant votre version de X-07. Il est impératif de bien noter cette adresse afin de pouvoir lancer LLIST par "EXEC adresse" dès que l'on en a besoin.

Le sous-programme relogeur, présenté dans les pages suivantes s'auto-détruit automatiquement dès qu'il a exécuté sa mission.

LLIST utilise le même principe d'analyse d'un programme BASIC que les logiciels REFBAS ou EXABAS : vous pouvez donc vous y reporter afin de bien cerner le processus de travail.

D'autre part, la concision du programme résulte de l'emploi de la routine &HFEFE incluse dans la ROM. Cette routine réécrit dans le tampon d'entrée (début : &HD5) la chaîne pointée par le registre HL (terminateur : code 0).

Longueur de LLIST : 554 octets

Implantation de LLIST : variable (suivant la configuration) .

La routine permettant de reloger LLIST est décrite dans les lignes suivantes. Ce programme permet des quantités d'applications en contrôlant parfaitement l'implantation de n'importe quelle routine.

Ce "relocateur" a été réalisé avec un ASSEMBLEUR autorisant le calcul sur les labels symboliques. Si vous ne disposez pas d'un tel outil, vous devez, postérieurement à l'assemblage, calculer et recréer la table des adresses à modifier.

D'autre part, lors de la construction de la table des adresses à modifier, il faut être particulièrement vigilant pour ne pas commettre un oubli irréparable !!

Enfin, il est inutile d'écrire "un programme relogeur" si le logiciel à "relocater" n'est pas parfaitement au point... En effet, toute modification du programme peut entraîner une ou des modifications de la table.

Voici le listing détaillé de cette routine :

```

ORG $800

DEB    LD HL,($212)      ; HL contient l'adresse du fond de la mémoire
      DEC HL            ;
      LD BC,fin-début    ; Longueur à déplacer
      XOR A             ; Mise du drapeau CARRY à 0
      SBC HL,BC          ; HL contient l'adresse du début du programme
      PUSH HL           ;
      PUSH HL           ; Empilages pour usages ultérieurs
      PUSH HL           ;
      CALL $CE9E         ; Effacement de l'écran
      LD HL,MSG1         ; Affichage sur l'écran LCD du message :
      CALL $FEF7         ; "NOTEZ AD=....."
      POP HL            ;
      CALL $BB98         ; Affichage de AD en décimal
      LD HL,MSG2         ; Affichage sur l'écran LCD du message :
      CALL $FEF7         ; "Pressez une touche"

KEY    XOR A             ;
      CALL $C90A         ; Attente de la pression d'une touche
      JR Z,KEY           ;
      POP HL            ; HL contient le début après transfert
      LD DE,début        ; DE contient le début après transfert
      XOR A             ;
      SBC HL,DE          ; HL = offset
      PUSH HL           ;
      POP BC            ; BC = offset
      LD IX,table        ; Table des adresses à modifier

DEB0   LD L,(IX+0)       ;
      LD H,(IX+1)       ; HL = adresse à modifier
      LD E,(HL)         ;
      INC HL            ;
      LD D,(HL)         ; DE = contenu à modifier
      DEC HL            ; Réajustement de HL
      PUSH HL           ; Sauvegarde de l'adresse à modifier
      EX DE,HL          ; HL = contenu à modifier
      ADD HL,BC          ; HL = HL + offset
      EX DE,HL          ; DE = valeur modifiée
      POP HL            ; Récupération de l'adresse à modifier
      LD (HL),E          ;
      INC HL            ; Modification effectuée
      LD (HL),D          ;
      INC IX            ; Adresse suivante de la table
      INC IX            ; Adresse suivante de la table
      LD A,(IX+0)        ;
      CP 0              ; Fini ?
      JR NZ,DEB0         ; Si ce n'est pas terminé , on boucle ...
      POP DE             ; ... Sinon DE = adresse début (MEM haute)
      LD HL,début        ; HL = adresse début (MEM basse)
      LD BC,fin-début    ; Longueur à transférer
      LDIR              ; Transfert du programme modifié
      LD HL,DEB          ;
      LD A,$C3           ; On écrit à la place de DEB les codes
      LD B,3             ; "C3C3C3" afin de rendre le relocateur
DEB1   LD (HL),A          ; inutilisable
      INC HL            ;
      DJNZ DEB1          ;
      JP $C3C3           ; Retour au BASIC
MSG1   DB 'NOTEZ AD=',0 ; 0 est le terminateur pour la routine FEF7

```

```

MSG2      DB 'Pressez ...',0 ; Message "Pressez une touche"
TABLE     DW A1+1,.....
          DW .....
          DB 0                ; Fin de la table

```

Exemple de programme à reloger :

```

DEBUT     CALL $CE9E
A1         LD HL,MSIMP        ;Première adresse à modifier
          CALL $FEF7
          .....
FIN        EQU $              ; Fin du logiciel à reloger

```

```

0800 : 2A 12 02 28 01 6C 01 AF *...(1..
0808 : ED 42 E5 E5 E5 CD 9E CE .B.....
0810 : 21 67 08 CD F7 FE E1 CD !g.....
0818 : 98 BB 21 71 08 CD F7 FE ..!q....
0820 : AF CD 0A C9 28 FA E1 11 ....(...
0828 : BD 08 AF ED 52 E5 C1 DD ....R...
0830 : 21 88 08 DD 6E 00 DD 66 !...n..f
0838 : 01 5E 23 56 2B E5 EB 09 .^#V+...
0840 : EB E1 73 23 72 DD 23 DD ..s#r.#.
0848 : 23 DD 7E 00 FE 00 20 E3 #.~... .
0850 : 01 21 B0 08 01 6C 01 ED .!...l..
0858 : B0 21 00 08 3E C3 06 14 .!...>...
0860 : 77 23 10 FC C3 C3 C3 4E w#.....N
0868 : 6F 74 65 7A 20 41 44 3D otez AD=
0870 : 00 0D 0A 0A 50 72 65 73 ....Pres
0878 : 73 65 7A 20 75 6E 65 20 sez une
0880 : 74 6F 75 63 68 65 2E 00 touche..
0888 : C1 08 D3 08 D6 08 E0 08 .....
0890 : E3 08 ED 08 FC 08 20 09 .....
0898 : 32 09 37 09 3F 09 44 09 2.7.?.D.
08A0 : 49 09 4D 09 53 09 58 09 I.M.S.X.
08A8 : 5B 09 5E 09 61 09 66 09 [.^.a.f.
08B0 : 6B 09 6F 09 72 09 7C 09 k.o.r.|.
08B8 : BC 09 C1 09 00 CD 9E CE .....
08C0 : 21 C5 09 C0 F7 FE AF CD !.....
08C8 : 0A C9 28 FA C0 9E CE CD ..(.....
08D0 : B7 CF 21 04 0A 11 0F 09 ..!.....
08D8 : 01 25 00 ED B0 3E 31 32 .%...>12
08E0 : 87 09 21 05 09 C0 F7 FE ..!.....
08E8 : CD F2 EB 23 11 0F 09 06 ...#....
08F0 : 14 7E FE 00 28 05 12 23 .~..(..#
08F8 : 13 10 F6 CD 56 09 DD 2A ....V...*
0900 : B2 00 AF DD BE 01 CA C3 .....
0908 : C3 DD E5 DD 6E 00 DD 66 ....n..f
0910 : 01 DD 5E 02 DD 56 03 E5 ..^..V..
0918 : DD E1 E1 01 04 00 09 CD .....
0920 : 90 09 06 06 C5 CD FE FE .....
0928 : C1 21 D5 00 7E FE 00 20 .!...~..
0930 : 05 CD 43 09 18 CC CD 93 ..C.....
0938 : 09 23 04 3E 27 B8 CC 43 .#.>'..C
0940 : 09 18 E9 CD 89 09 06 00 .....
0948 : 3A 88 09 3C 32 88 09 FE :...<2...
0950 : 38 C0 CD 56 09 C9 E5 21 8..V...!
0958 : 81 09 CD 77 09 21 DF 09 ...w.!..
0960 : CD 77 09 3E 02 32 88 09 .w.>.2..
0968 : AF 47 3A 87 09 3C 32 87 .G:...<2.
0970 : 09 21 FE 09 77 E1 C9 7E .!...w..~
0978 : FE 00 C8 CD 93 09 23 18 .....#.

```

FIGURE 21 : LLIST

```

0980 : F6 0D 0A 0A 0A 0A 00 31 .....1
0988 : 02 E5 D5 C5 CD B0 CF C1 .....
0990 : D1 E1 C9 E5 D5 C5 CD F7 .....
0998 : CE C1 D1 E1 C9 E5 E8 22 .....
09A0 : 50 04 01 07 07 21 D5 00 P....!..
09A8 : CD 5F BE 21 D5 00 3E 30 ._.!...>0
09B0 : BE 20 05 36 20 23 18 F6 . .6 #..
09B8 : 21 D5 00 CD 77 09 3E 20 !...w.>
09C0 : CD 93 09 E1 C9 49 6D 70 .....Imp
09C8 : 72 69 6D 61 6E 74 65 20 rimante
09D0 : 4F 4B 20 3F 00 54 69 74 OK ?.Tit
09D8 : 72 65 20 3A 0A 0D 00 20 re :...
09E0 : 20 20 20 20 20 20 20 20
09E8 : 20 20 20 20 20 20 20 20
09F0 : 20 20 20 20 20 20 20 20
09F8 : 20 20 20 20 20 20 20 20
0A00 : 20 20 20 20 20 20 20 20
0A08 : 20 20 20 20 20 20 20 20
0A10 : 20 20 20 20 20 20 20 20
0A18 : 20 20 20 20 20 50 61 67 Pag
0A20 : 65 3A 20 31 20 0D 0A 0A e: 1 ...
0A28 : 00 A5 2E 2E 2C 2E 2E 2E .....,...

```

FIGURE 21 : FIN

```

10 CLEAR 50,&H7FF
20 INIT#1,"CASI:"
30 INPUT#1,N$,D,F
40 MOTOR
50 PRINT"Trouv :";N$
60 FOR I=D-1 TO F
70 POKE I,INP(#1)
80 NEXT: MOTOR
90 END
100 CLEAR 50,&H7FF
110 D=&H800 : F=&HA29
120 N$="LLIST": INIT#1,"CASO:"
130 INPUT"Magnto OK";T$
140 PRINT#1,N$,D,F: MOTOR
150 FOR I=1 TO 1800: NEXT
160 FOR I=D TO F: OUT#1,PEEK(I)
170 NEXT: MOTOR
180 END

5 ' *** ENTREUR DE CODES ***
10 CLEAR 50,&H7FF: A=&H800
20 PRINT HEX$(A);" : ";: INPUT C$
30 V=VAL("&H"+C$): POKE A,V
49 A=A+1: IF A>&HA29 THEN PRINT "...":
BEEP 2,3: END
50 GOTO 20

```

FIGURE 22 : CHARGEURS

LISTING LOGO CANON Page: 1

```

10 CLEAR 200: DEFINT A-Z
20 X=23: CLS
30 FOR L=6 TO 25
40 A1=1: F=1
50 READ A: A1=A1+A: IF F=1 THEN 70
60 F=1: GOTO 80
70 FOR I=A1-A TO A1-1: PSET(X+I,L):
NEXT: F=0
80 IF A1=<79 THEN 50
90 NEXT
100 END
110 DATA 79,9,4,66,8,6,65,6,10,63
120 DATA 5,12,7,4,9,3,3,2,7,7,6,3,
3,2,6
130 DATA 4,6,6,2,5,6,7,5,1,4,5,9,4
,5,1,4,5
140 DATA 4,5,6,4,2,9,4,13,3,2,3,6,
1,13,4
150 DATA 3,6,6,2,10,3,6,5,2,5,2,3,
2,6,3,5,2,5,3
160 DATA 3,5,6,1,12,4,5,5,2,5,1,4,
3,6,2,5,2,5,3
170 DATA 3,5,15,8,5,5,2,5,1,4,3,6,
2,5,2,5,3
180 DATA 3,5,14,10,4,5,2,5,1,5,3,5
,2,5,2,5,3
190 DATA 3,5,13,5,2,4,4,5,2,5,1,5,
3,5,2,5,2,5,3
200 DATA 3,6,11,5,4,4,3,5,2,5,1,6,
3,4,2,5,2,5,3
210 DATA 4,5,9,1,1,5,4,4,3,5,2,5,1
,6,3,4,2,5,2,5,3
220 DATA 4,6,6,2,2,6,2,6,2,5,2,5,2
,6,2,3,3,5,2,5,3
230 DATA 5,12,4,8,1,4,2,5,2,5,2,6,
3,2,3,5,2,5,3
240 DATA 6,10,6,6,2,5,1,5,2,5,3,9,
4,5,2,5,3
250 DATA 8,6,9,4,3,5,1,5,2,5,4,7,5
,5,2,5,3,79,79

```

FIGURE 23 : EXEMPLE

Labyrinthe 3D

Ce jeu n'est pas unique en son genre mais il va vous faire passer d'agréables moments grâce à sa rapidité et à ses multiples options.

Le but est de se sortir d'un labyrinthe compliqué dont vous pouvez obtenir deux vues différentes : une vue de "haut" (totalité du dédale visible) ou une vue de "bas" (vue en perspective de la partie où vous vous trouvez...). La boussole affichée sur l'écran vous aidera à vous en sortir... Du moins, nous l'espérons !!

D'autre part , un score est affiché sur l'écran tout au long de la partie... En effet, vous devez mettre le moins de temps possible à sortir du labyrinthe pour espérer obtenir le score le plus bas possible.

Le mode d'emploi est simple :

Après avoir chargé le logiciel (Soit CLOAD puis RUN , soit avec le chargeur de codes...), faire EXEC &H800.

Une petite présentation défile : tapez sur une touche pour continuer.

De succinctes explications apparaissent accompagnées des commandes disponibles : Nord, West, Est, Sud pour les directions, B pour retourner au BASIC, R pour repartir du point de départ, A pour avancer dans le dédale et L pour avoir une vue d'ensemble... Cette dernière fonction coûte 20 points : usez-en avec parcimonie !!

Après avoir tapé sur une touche quelconque, le labyrinthe se crée automatiquement... Dès que vous êtes prêt à y pénétrer, pressez une touche...

Vous vous trouvez alors à l'entrée du labyrinthe... La sortie se trouve à l'opposé ! Rassurez-vous, le minotaure est en vacances !

Longueur de LABYRYNTHE : 2066 octets .

Implantation de LABYRINTHE : de 800h à 1012h

Quelques indications sur l'algorithme de création du dédale :

1. Créer un tableau de cellules vides.
2. Choisir une origine voisine du centre.
3. Tester les quatre cases voisines afin de déterminer celles qui ont été visitées.
4. Si toutes les cases voisines ont été visitées , ajuster les pointeurs jusqu'à retomber dans une case déjà visitée , limitrophe d'une case non visitée.
5. Choisir aléatoirement parmi les cellules limitrophes non visitées.
6. Y aller en créant une porte dans la cellule de départ et une dans la cellule d'arrivée . Marquer la cellule d'arrivée.
7. Si toutes les cellules du tableau n'ont pas été testées , remonter en 3/ sinon créer une entrée et une sortie.

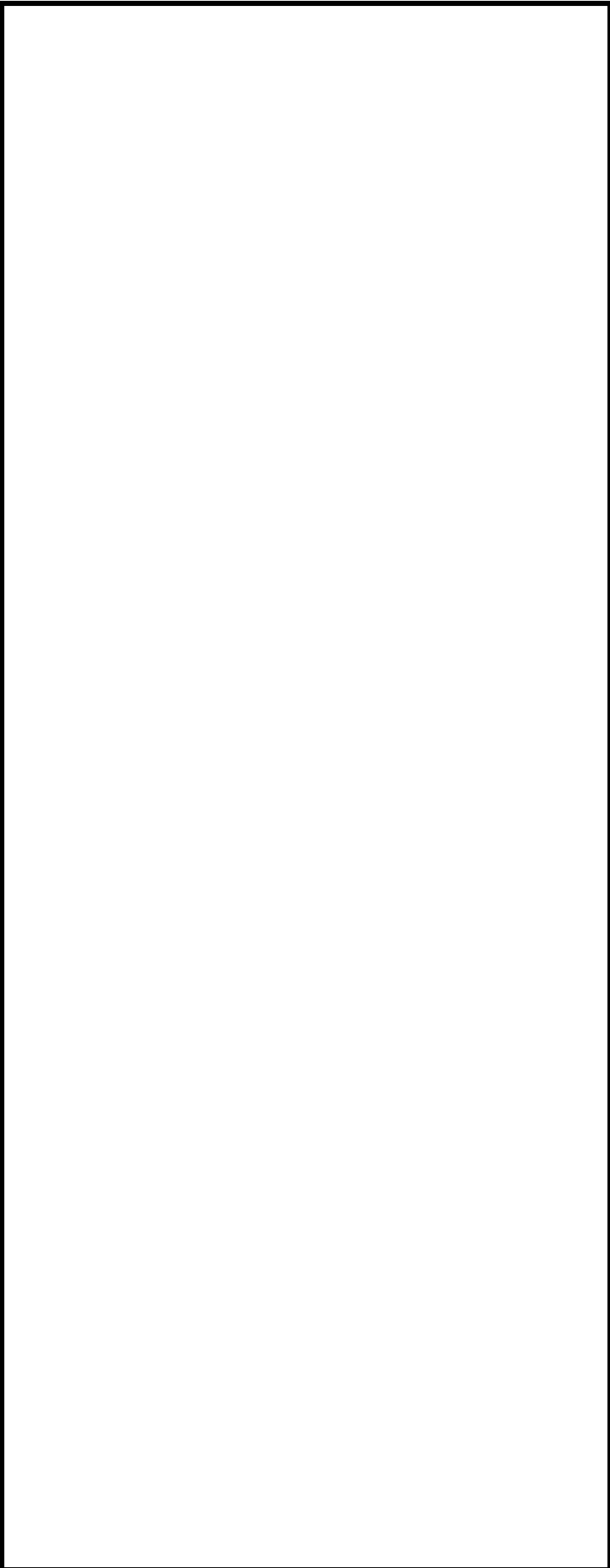


FIGURE 24 : LABYRINTHE 3D



FIGURE 24 : SUITE

FIGURE 24 : SUITE

FIGURE 24 : SUITE

```

2 '*** PROGRAMME 1 ***
5 '*** CHARGEUR BASIC ***
10 CLEAR 50,&H7F0
20 INIT#1,"CASI:"
30 INPUT#1,N$,D,F
40 MOTOR
50 PRINT"Trouv :";N$
60 FOR I=D-1 TO F
70 POKE I,INP(#1)
80 NEXT: MOTOR
90 END
100 CLEAR 50,&H7F0
110 D=&H800: F=&H1012
120 N$="PLABIR": INIT#1,"CASO:"
130 INPUT"Magnto OK";T$
140 PRINT#1,N$,D,F: MOTOR
150 FOR I=1 TO 1800: NEXT
160 FOR I=D TO F: OUT#1,PEEK(I)
170 NEXT: MOTOR
180 END

2 '*** PROGRAMME 2 ***
5 ' *** ENTREUR DE CODES ***
10 CLEAR 50,&H7FF: A=&H800
20 PRINT HEX$(A);" : ";: INPUT C$
30 V=VAL("&H"+C$): POKE A,V
49 A=A+1: IF A>&H1012 THEN PRINT "...":
BEEP 2,3: END
50 GOTO 20

```

FIGURE 24 : FIN

FIGURE 25 : CHARGEURS

Le Solitaire

Pas un jeu n'est à la fois plus familier et plus méconnu que le SOLITAIRE. Ses règles sont si simples qu'il n'a pas la réputation d'être un jeu de réflexion. Néanmoins, il exclut toute forme de hasard, ce qui en fait un excellent casse-tête.

Le matériel qu'il requiert est des plus simples : il se réduit à une planchette percée de 33, 37 ou 41 trous dans lesquels viennent se loger autant de fiches, billes ou coquillages. Le solitaire à 33 trous , aussi appelé "Solitaire ANGLAIS", connut un grand succès en Angleterre et en Allemagne. Le logiciel proposé utilise d'ailleurs le type anglais. Notons tout de même que deux autres versions existent : le "solitaire FRANCAIS" à 37 cases et le "GRAND Solitaire" à 41 trous.

Les règles du jeu sont des plus faciles à assimiler. Il suffit de mettre en place toutes les fiches dans les trous prévus à cet effet, d'en ôter une au choix puis de commencer à les retirer comme on le fait au jeu de DAMES. Remarquons qu'une fiche peut prendre une autre fiche contiguë en sautant par-dessus, horizontalement ou verticalement, à condition de retomber dans un trou inoccupé.

Si aucun mouvement ne peut être effectué, la partie est terminée... Plus le nombre de fiches est restreint sur le damier, meilleur est le score, l'idéal étant qu'une seule fiche demeure.

Le mode d'emploi est des plus simples :

Après avoir chargé le programme dans la mémoire du X-07 par CLOAD puis RUN (ou par le chargeur de codes si vous ne possédez pas la K7...), faire EXEC &H800.

L'option 1 du menu permet d'accéder à une recherche par le X-07. L'utilisateur choisit le nombre de pions de la solution finale et le programme livrera une solution après un laps de temps plus ou moins long , suivant l'option choisie. En effet, le canoniste a le choix entre 22 possibilités de recherche.

Aucun algorithme simple n'est connu pour ce jeu et le programme effectue une recherche systématique. A chaque étape, il examine successivement toutes les cases dans les quatre directions possibles... Dès qu'un coup est jouable, le X-07 l'exécute. Si la recherche conduit à une impasse, le CANON remonte niveau par niveau : l'ordre d'examen des directions possède donc une importance capitale en ce qui concerne la vélocité avec laquelle le logiciel parvient à une solution.

L'option 2 du menu donne accès au jeu du joueur : le programme assure l'affichage du damier, enregistre, teste et exécute les coups proposés par le joueur. Plusieurs commandes sont disponibles pour jouer :

- Déplacer le curseur sur la case de départ au moyen des flèches et presser sur la touche "D": un marqueur s'allumera...
- Déplacer ensuite le curseur sur la case d'arrivée et presser la touche "A" afin d'exécuter le coup.
- La touche "-" permet de revenir au coup précédent.
- La touche "Q" permet d'arrêter le jeu quand il est bloqué.
- La commande "CTRL Q" redonne "la main" au BASIC.

Si le nombre de pions est supérieur ou égal à 3 dans l'option 1, le logiciel trouve une réponse en quelques secondes. Par contre, si le nombre de pions est égal à 2, l'attente peut varier de cinq minutes à une heure selon le choix de l'ordre. Pour l'ordre 9, le programme trouve une solution classique en moins de dix minutes.

Enfin, le logiciel affiche la profondeur d'analyse dans l'arbre. Cette opération se révèle très couteuse en temps d'exécution... Pour la supprimer, il suffit de mettre à 0 les octets &H989, &H98A et &H98B par l'ordre POKE du BASIC, par exemple.

Voici, pour terminer, un schéma du Solitaire anglais :

		1	2	3		
		4	5	6		
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
		28	29	30		
		31	32	33		

Longueur de SOLITAIRE : 4 609 octets.

Implantation de SOLITAIRE : de 800h à 11D5h.

FIGURE 26 : SOLITAIRE

FIGURE 26 : SUITE

FIGURE 26 : SUITE

FIGURE 26 : SUITE

```

10 CLEAR 50,&H7FF
20 INIT#1,"CASI:"
30 INPUT#1,N$,D,F
40 MOTOR
50 PRINT"Trouv :";N$
60 FOR I=D-1 TO F
70 POKE I,INP(#1)
80 NEXT: MOTOR
90 END
100 CLEAR 50,&H7FF
110 D=&H800: F=&H11D5
120 N$="SOL": INIT#1,"CASO:"
130 INPUT"Magnto OK";T$
140 PRINT#1,N$,D,F: MOTOR
150 FOR I=1 TO 1800: NEXT
160 FOR I=D TO F: OUT#1,PEEK(I)
170 NEXT: MOTOR: END

```

```

5 REM *** ENTREUR DE CODES ***
10 CLEAR 50,&H7FF:A=&H800
20 PRINTEX$(A);" : ";;INPUT C$
30 V=VAL("&H"+C$):POKE A,V
49 A=A+1:IF A>&H11D5 THEN
PRINT"...":BEEP 2,3:END
50 GOTO20

```

FIGURE 26 : SUITE

FIGURE 27 : CHARGEURS

Les Pentominos

Jouer aux PENTOMINOS, c'est s'attaquer à toute une série de puzzles bien particuliers. Des constructions à manier avec votre sens des mathématiques, de la logique... et de la stratégie.

En fait, pour les non-initiés, les pentominos constituent les formes obtenues en juxtaposant le long de leurs côtés plusieurs carrés de surface identique. Le casse-tête utilisant les douze pentominos est assez ancien : on en trouve des énoncés dès 1907 dans des revues spécialisées telles que "CANTERBURY PUZZLES".

Il existe donc douze manières différentes d'assembler cinq carrés en les joignant par leurs arêtes : les pentominos sont constitués. Pour les identifier, on utilise leur ressemblance avec l'alphabet.

Le problème du rangement des pentominos dans des cadres rectangulaires a été résolu. On sait qu'il n'existe que 2 solutions pour un cadre 3×20 , 368 pour un cadre 4×15 , 1010 pour un cadre 5×12 et 2339 pour un cadre 6×10 .

Malgré le grand nombre de solutions possibles et le petit nombre de pièces, la résolution de ce puzzle est difficile.

Le programme présenté a été écrit pour effectuer une résolution automatique de ce puzzle : il procède par "essais-erreurs" jusqu'à ce qu'une solution soit trouvée. Pour obtenir des solutions différentes, l'ordre d'examen des pièces est modifiable... Notons que cet ordre conditionne la durée du calcul.

Si une imprimante X-710 est reliée au X-07, il est possible d'imprimer les résultats obtenus.

Le mode d'emploi est très simple :

- Possesseurs de la K7, faites CLOAD puis RUN pour charger les codes. Sinon, rentrez-les à la main avec les chargeurs fournis.
- Le programme est lancé avec EXEC &H800.
- Tapez sur une touche après chaque affichage. Choisissez votre tableau et l'ordre d'examen de pièces.
- Le CANON recherche la solution et vous la donne après quelques instants de réflexion. Vous pouvez alors l'imprimer sur la X-710.
- Pour stopper le jeu et retourner au BASIC, taper sur "CTRL Q".
- Si vous désirez une solution rapide pour les trois damiers, vous pouvez entrer la combinaison de recherche "TFILUVNWPXSY".

Voici, pour terminer, le schéma des pièces :

[DESSINS TO DO]

Longueur de PENTOM INOS : 3489 octets.

Implantation de PENTOMINOS : de 800h à 159Ah.

FIGURE 28 : PENTOMINOS

FIGURE 28 : SUITE

FIGURE 28 : SUITE

FIGURE 28 : SUITE

FIGURE 28 : SUITE

FIGURE 28 : SUITE

FIGURE 28 : SUITE

FIGURE 28 : SUITE


```

10 CLEAR 50,&H7FF
20 INIT#1,"CASI:"
30 INPUT#1,N$,D,F
40 MOTOR
50 PRINT"Trouv :";N$
60 FOR I=D-1 TO F
70 POKE I,INP(#1): NEXT: MOTOR: END
100 CLEAR 50,&H7FF
110 D=&H800: F=&H159A
120 N$="PENTA": INIT#1,"CASO:"
130 INPUT"Magnto OK";T$
140 PRINT#1,N$,D,F: MOTOR
150 FOR I=1 TO 1800: NEXT
160 FOR I=D TO F: OUT#1,PEEK(I)
170 NEXT: MOTOR: END

```

```

5 REM *** ENTREUR DE CODES ***
10 CLEAR 50,&H7FF:A=&H800
20 PRINTEX$(A);" : " : INPUT C$
30 V=VAL("&H"+C$):POKE A,V
49 A=A+1:IF A>&H159A THEN PRINT"...":
BEEP 2,3: END
50 GOTO20

```

FIGURE 28 : SUITE

FIGURE 29 : CHARGEURS

Autonum

Ce programme implémente la fonction AUTO... Ce qui signifie que le CANON va disposer d'une numérotation automatique des lignes de programme BASIC !!

Après avoir chargé le programme "AUTONUM" dans la mémoire du X-07 il suffit de choisir l'adresse de fin de la routine à implanter suivant la configuration de votre CANON. Ensuite, un FSET judicieux protégera votre routine (FSET 215 minimum).

La procédure d'implantation de la routine résidente va pouvoir être débutée. Pour cela, faire RUN suivi de RETURN... Le logiciel vous demande votre adresse de fin (en hexadécimal) et l'adresse de lancement vous est renvoyée. Le chargeur devenu inutile peut être détruit par l'ordre NEW.

L'utilisation d'AUTONUM est la suivante :

- En tapant CTRL N, le programme affiche un point d'interrogation. Vous devez alors entrer le numéro de la première ligne et l'incrément choisi. Exemple: en tapant, "100,10" suivi de RETURN, la séquence générée sera 100, 110, 120, 130...
- L'appui sur CTRL Z agit comme une bascule permettant d'entrer et de ressortir de la fonction AUTO.
- CTRL Q désactive de manière définitive AUTONUM. Pour le relancer, il faut alors taper EXEC AD, AD représentant l'adresse de lancement affichée au tout début des opérations.

AUTONUM fonctionne grâce à la modification de l'interruption clavier. Si une pression est détectée sur la touche RETURN, le logiciel agit de la manière suivante :

- Exécution de l'opération demandée.
- Si AUTONUM est actif, écrire dans le tampon d'entrée le numéro de la ligne et l'incrément (pour la ligne suivante).
- Revenir à l'exécution normale.

Voici quelques remarques sur la programmation de AUTONUM...

Tout d'abord, ce logiciel utilise cinq octets de mémoire morte pour stocker ses données... Ces octets sont situés dans le tampon utilisé par les routines CSAVE et CLOAD.

Ensuite , AUTONUM agit par interception du vecteur des interruptions (&H3D). Or le programme est relogeable : il faut donc trouver une astuce pour que ce logiciel effectue de lui-même la modification correcte de ce vecteur. Une solution est décrite ci-dessous :

Début	LD HL,\$D5D1	; Ecriture en \$300 de la séquence :
	LD (\$300),(HL)	; POP DE
	LD A,\$C9	; PUSH DE
	LD (\$302),A	; RET
	CALL \$300	; Lors du CALL , l'adresse XXX est empilée
XXX	LD HL,14	; Au retour du CALL , DE = XXX
	ADD HL,DE	; HL = AUTONUM (14 octets entre XXX et AUTONUM)
	DI	;
	LD (\$3D),HL	; Nouveau vecteur d'interruption
	EI	;
	XOR A	; Inactiver la routine AUTONUM : l'adresse
	LD (\$300),A	; \$300 contient le drapeau activé par la
	RET	; séquence CTRL Z
AUTO	Début du logiciel ...	

Longueur de AUTONUM (BASIC) : 1 093 octets.

Longueur de la routine : 215 octets.

Implantation de la routine : relogeable.

```

1 REM fonction AUTONUM pour X-07
10 CLEAR 50,&HFFF: DEFINT A-Z: CLS
20 INPUT"Adr. fin ";A$: F=VAL("&H"+A$)
30 D=F-215: PRINT"Adr. deb = ";HEX$(D)
40 FOR I=D TO F: READ A: POKE I,A: NEXT
50 EXEC D: END
10000 DATA 033,209,213,034,000,003,062,
201,050,002,003,205,000,003,033,014
10010 DATA 000,025,243,034,061,000,251,
175,050,000,003,201,217,008,219,242
10020 DATA 230,001,202,193,200,219,240,
230,192,040,008,230,128,202,053,200
10030 DATA 195,012,200,219,241,254,013,
032,075,095,175,205,098,194,175,205
10040 DATA 170,194,058,000,003,183,202,
189,200,062,001,211,245,217,008,197
10050 DATA 213,229,245,237,091,001,003,
042,003,003,025,034,001,003,235,205
10060 DATA 156,187,035,205,110,213,205,
004,215,205,059,202,010,197,213,095
10070 DATA 175,205,098,194,175,205,170,
194,209,193,021,003,032,238,241,225
10080 DATA 209,193,251,201,254,026,032,
027,062,001,211,245,217,008,197,213
10090 DATA 229,245,062,013,239,062,010,
239,033,000,003,175,182,047,119,032
10100 DATA 221,024,176,254,014,032,041,
062,001,211,245,217,008,197,213,229
10110 DATA 245,175,050,000,003,205,242,
235,215,205,204,255,237,083,001,003
10120 DATA 207,044,205,204,255,237,083,
003,003,175,061,050,000,003,024,131
10130 DATA 254,017,194,128,200,195,195,
195

```

FIGURE 30 : AUTONUM

REFBAS

REFBAS est un utilitaire qui permet d'obtenir soit une liste triée de toutes les variables d'un programme BASIC, soit une table croisée des références à un numéro de ligne. Ce logiciel est donc particulièrement utile si l'on désire analyser, modifier les noms des variables, réorganiser ou documenter un programme BASIC, surtout si ce dernier est long.

Le mode d'emploi de REFBAS est le suivant :

- Avant de charger REFBAS, faire FSET 1024. Cette opération permettra la protection ultérieure de REFBAS situé en fin de mémoire fichier (version 16Ko). Carte moniteur en place : FSET 4096.
- Charger REFBAS en tapant CLOAD suivi de RETURN puis RUN toujours suivi de RETURN. Les codes se chargeront et le programme sera lancé par la commande EXEC &H3C10. Comme toujours, si vous ne possédez pas la K7, les chargeurs sont à votre disposition.
- Un menu s'affiche alors avec trois options disponibles :
 - OPTION V :REFBAS construit une table triée de toutes les variables et des numéros de lignes où ces variables sont appelées
 - OPTION S :le programme édifie une table des références croisées pour les instructions GOTO, GOSUB, THEN, ELSE, RESUME, RUN et RESTORE. Le numéro des lignes ciblées est imprimé en tête ainsi que le numéro des lignes faisant référence à cette cible
 - OPTION B : retour au BASIC.
- Remarquons que la commande CTRL Q permet de revenir au BASIC à n'importe quel moment et que BREAK est actif durant la phase d'impression.

Pendant son fonctionnement, REFBAS crée à la suite du programme à analyser une table utilisant six octets par variable ou référence. Quand cette table arrive au contact de la pile, l'utilitaire annonce que la taille mémoire est insuffisante : il convient dans ce cas d'analyser le programme étudié en le segmentant.

Pour terminer, voici une petite note technique pour les fans de la programmation avancée...

La longueur des lignes (40 pour la X-710) est contenue à l'adresse &H3EDB. Si vous désirez imprimer en plus petit, libre à vous !!

Tous les BASIC issus de la firme MICROSOFT' sont structurés de la même manière :

- 2 octets de chaînage indiquant l'adresse absolue de la ligne suivante.
- 2 octets pour le numéro de ligne.
- n octets de code . Les mots clés du BASIC sont codés sur un ou deux octets (un seul pour le X-07) afin de limiter l'encombrement mémoire et de faciliter la tâche de l'interpréteur. Les autres caractères sont codés en ASCII.
- 1 octet égal à 0 représentant l'indicateur de fin de ligne.

L'adresse de la zone programme est contenue dans l'octet &HB2 (indicateur TXTTAB) et est égal, en principe, à &H553. Vous pouvez utilement consulter les "Mystères du X-07" pour plus de précisions.

La boucle de scrutation du programme BASIC de REFBAS (similaire à celle d'EXABAS et LLIST) est structurée de la manière suivante :

```
BOUCLE      LD IX,(TXTTAB)      ; Initialisation
            XOR A              ; A=0
            CP (IX+1)          ; Si (IX+1)=0 , c'est la fin du programme
            JP Z,fin           ; Fin
            PUSH IX            ; Sauve l'adresse du début de la ligne
            LD L,(IX+0)        ; L'adresse de la ligne suivante est stockée
            LD H,(IX+1)        ; dans le registre HL
            LD E,(IX+2)        ; Le numéro de ligne est stocké dans le
            LD D,(IX+3)        ; registre DE
            PUSH HL            ;
            POP IX             ; IX pointe sur la ligne suivante
            POP HL             ; On récupère l'adresse du début de la ligne
            LD BC,4            ;
            ADD HL,BC          ; HL contient le premier code de la ligne
            ...                ; Traitement
            JP BOUCLE          ;
```

REFBAS se livre à une analyse détaillée des différents types de variables et envisage tous les problèmes de chaîne. En ce qui concerne les références , il va les chercher derrière les mots clés.

Longueur de REFBAS : 985 octets.

Implantation de REFBAS : de 3C10 à 3FE9h.

FIGURE 31 : REFBAS

FIGURE 31 : SUITE

```

10 CLEAR 50,&H7FF
20 INIT#1,"CASI:"
30 INPUT#1,N$,D,F
40 MOTOR
50 PRINT"Trouv :";N$
60 FOR I=D-1 TO F
70 POKE I,INP(#1)
80 NEXT: MOTOR
90 END
100 CLEAR 50,&H7FF
110 D=&H3C10: F=&H3FE9
120 N$="REFBAS": INIT#1,"CASO:"
130 INPUT"Magnto OK";T$
140 PRINT#1,N$,D,F: MOTOR
150 FOR I=1 TO 1800: NEXT
160 FOR I=D TO F: OUT#1,PEEK(I)
170 NEXT: MOTOR
180 END

```

```

5 REM *** ENTREUR DE CODES ***
10 CLEAR 50,&H9FF:A=&H3C10
20 PRINTHEX$(A);" : ";;INPUT C$
30 V=VAL("&H"+C$):POKE A,V
49 A=A+1:IF A>&H3FE9 THEN
PRINT"...":BEEP 2,3:END
50 GOTO20

```

ACTION DE REFBAS SUR LE CHARGEUR

Liste des variables

D	30 60 110 140 160
F	30 60 110 140 160
I	30 70 150 160 160
N\$	30 50 120 140
T\$	130

FIGURE 31 : FIN

FIGURE 32 : EXEMPLE+CHARGEURS

EXABAS

Dans la continuité de REFBAS, voici EXABAS qui constitue aussi un utilitaire puissant en implémentant les fonctions REF (références) et SR (Search and Replace) que l'on trouve sur certains gros ordinateurs dotés de fonctions d'éditeur infiniment plus complètes et plus performantes...

EXABAS constitue donc un utilitaire permettant de trouver dans un programme BASIC toutes les occurrences d'une variable, d'une d'un mot-clé. Il permet également de remplacer une chaîne quelconque ou un mot-clé par une autre chaîne ou un autre mot-clé.

Le mode d'emploi d'EXABAS est aussi simple que celui de REFBAS :

- Avant de charger EXABAS, faire FSET 1024 (ou FSET 4096 si une carte moniteur est en place) pour le protéger.
- Charger EXABAS de la même manière que REFBAS.
- Lancer EXABAS par la commande EXEC &H3C10.
- Un menu comportant six options s'affiche alors :
 - OPTION B : sortie du programme et retour au BASIC.
 - OPTION V : recherche d'une variable particulière.
 - OPTION C : recherche d'une chaîne quelconque (sauf les mots-clés...).
 - OPTION I : recherche d'un mot-clé quelconque.

Pour ces précédentes options, EXABAS réclame la chaîne à identifier puis affiche les numéros de ligne où cette chaîne figure.

- OPTION c : remplacement d'une chaîne par une autre.
- OPTION t : remplacement d'un mot-clé par un autre.

Pour ces deux dernières options, EXABAS réclame le nom de la chaîne à remplacer puis celle de remplacement. Il affiche ensuite les numéros des lignes où l'opération est effectuée.

- L'appui sur CTRL Q permet de revenir instantanément au BASIC.
- Dans tous les cas, le logiciel marque une pause quand l'afficheur est rempli. Il attend la pression sur une touche pour continuer l'exécution. Si cette touche est "I", le contenu de l'écran sera envoyé vers la X-710. Notons qu'une fois l'analyse terminée, le X-07 affiche un carreau.

EXABAS permet quantité d'applications. C'est un outil puissant qui a été testé sans problème sur de nombreux programmes. Néanmoins, il est indispensable d'effectuer une copie de protection des programmes à étudier avant d'utiliser EXABAS.

Au niveau technique, les problèmes de recherche sont traités de la même manière que REFBAS. Le problème du remplacement des chaînes est plus délicat à traiter car la longueur du programme est susceptible de varier. Aussi, après chaque modification, est-il nécessaire d'ajuster tous les pointeurs des lignes suivant celle qui vient d'être modifiée.

Longueur d'EXABAS : 901 octets

Implantation d'EXABAS : de 3C10h à 3F95h.

FIGURE 33 : EXABAS

FIGURE 33 : SUITE

```

10 CLEAR 50,&H9FF
20 INIT#1,"CASI:"
30 INPUT#1,N$,D,F
40 MOTOR
50 PRINT"Trouv :";N$
60 FOR I=D-1 TO F
70 POKE I,INP(#1)
80 NEXT: MOTOR
90 END
100 CLEAR 50,&H9FF
110 D=&H3C10: F=&H3F95
120 N$="EXABAS": INIT#1,"CASO:"
130 INPUT"Magnto OK";T$
140 PRINT#1,N$,D,F: MOTOR
150 FOR I=1 TO 1800: NEXT
160 FOR I=D TO F: OUT#1,PEEK(I)
170 NEXT: MOTOR
180 END

```

```

5 REM *** ENTREUR DE CODES ***
10 CLEAR 50,&H9FF:A=&H3C10
20 PRINTHEX$(A);" : ";:INPUT C$
30 V=VAL("&H"+C$):POKE A,V
49 A=A+1:IF A>&H3F95 THEN
PRINT"...":BEEP 2,3:END
50 GOTO20

```

ACTION DE REFBAS SUR LE CHARGEUR

Liste des variables

Nom? N	30 50 120 140
Mot Clé? PRINT	50 140
Mot Clé? DEFSTR	
Nom? T\$	130

FIGURE 33 : FIN

FIGURE 34 : CHARGEURS

Le Piège

Enfin un vrai jeu d'aventures pour le CANON X-07 !!! Ce genre de jeu était vraiment très rare pour ne pas dire inexistant et nous avons décidé de vous y faire goûter... Vous n'allez pas être déçu !

Le but du jeu est d'explorer une grande maison : la visite de toutes les salles est indispensable pour atteindre la sortie... Bonne chance !

Le mode d'emploi est le suivant :

Charger le programme en version vidéo ou LCD suivant vos désirs. Comme toujours . le petit programme BASIC se chargera de rentrer les codes en mémoire. Faire un FSET avant le chargement tout en sachant que la mémoire est altérée de &H800 à &H1900.

le programme possède trois points d'entrée :

- EXEC &H800 : début du jeu .
- EXEC &H815 : vous repartez de l'endroit quitté par la commande "P"
- EXEC &H838 : vous repartez de l'endroit quitté par la commande "Q"

Le programme affiche sur la ligne numéro :

- 1 : le nom de la salle et les directions possibles (N, S, E, W)
- 2 : les objets mobiles éventuellement présents dans cette salle
- 3 : un message de description de la salle
- 4 : ligne utilisée pour recevoir vos commandes et afficher les réponses du programme

Les commandes se répartissent en deux catégories :

- composées d'une lettre
 - N , S , E , W pour se déplacer
 - I pour avoir la liste des objets possédés
 - Q pour quitter
 - P pour faire une pause (sauvegarde)
- Normales :
 - le programme comprend des "phrases" composées d'un verbe à l'infinitif suivi d'un nom. Ex : "SAUTER COULOIR"

- En fait , seules les quatre premières lettres de chaque mot sont analysées et il suffit de taper dans l'exemple " S AUT COUL". Si le verbe n'est pas compris , " ????" est affiché. Par contre . si le nom n'est pas compris , " ????" apparait

Longueur des logiciels : 4205 octets (LCD) et 4329 octets (vidéo)

Implantation : de 800h à 186Ch (LCD) / de 800h à 1 8E8h (vidéo)

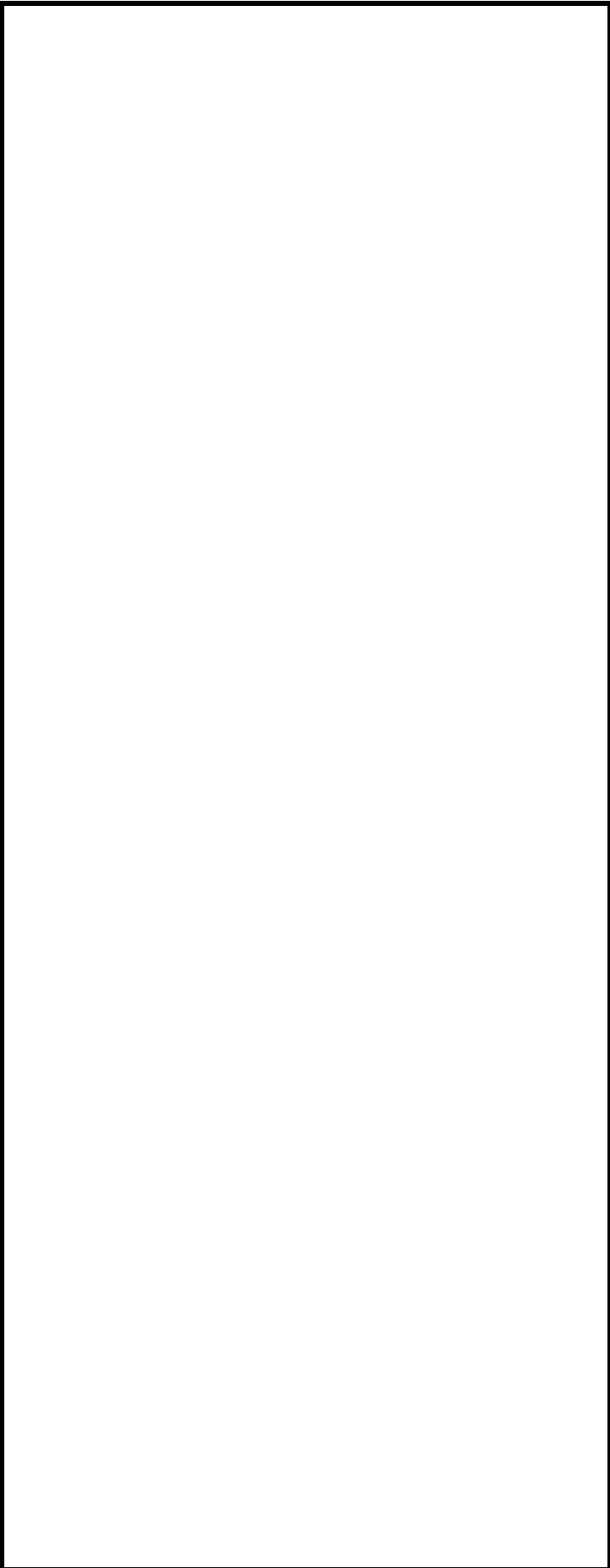


FIGURE 35 : LE PIEGE (LCD)



FIGURE 35 : SUITE

FIGURE 35 : SUITE

FIGURE 35 : SUITE

FIGURE 35 : SUITE

FIGURE 35 : SUITE

FIGURE 35 : SUITE

FIGURE 35 : SUITE

FIGURE 35 : SUITE

FIGURE 35 : SUITE

```

5 REM *** CHARGEUR - VERSION LCD ***
10 CLEAR 50,&H7FF
20 INIT#1,"CASI:"
30 INPUT#1,N$,D,F
40 MOTOR
50 PRINT"Trouv :";N$
60 FOR I=D-1 TO F
70 POKE I,INP(#1)
80 NEXT: MOTOR
90 END
100 CLEAR 50,&H9FF
110 D=&H800: F=&H186C
120 N$="AVENT2": INIT#1,"CASO:"
130 INPUT"Magnto OK";T$
140 PRINT#1,N$,D,F: MOTOR
150 FOR I=1 TO 1800: NEXT
160 FOR I=D TO F: OUT#1,PEEK(I)
170 NEXT: MOTOR
180 END

```

```

5 REM *** ENTREUR DE CODES - LCD ***
10 CLEAR 50,&H7FF:A=&H800
20 PRINTHEX$(A);" : ";:INPUT C$
30 V=VAL("&H"+C$):POKE A,V
49 A=A+1:IF A>&H186C THEN
PRINT"...":BEEP 2,3:END
50 GOTO20

```

FIGURE 35 : SUITE

FIGURE 36 : CHARGEURS

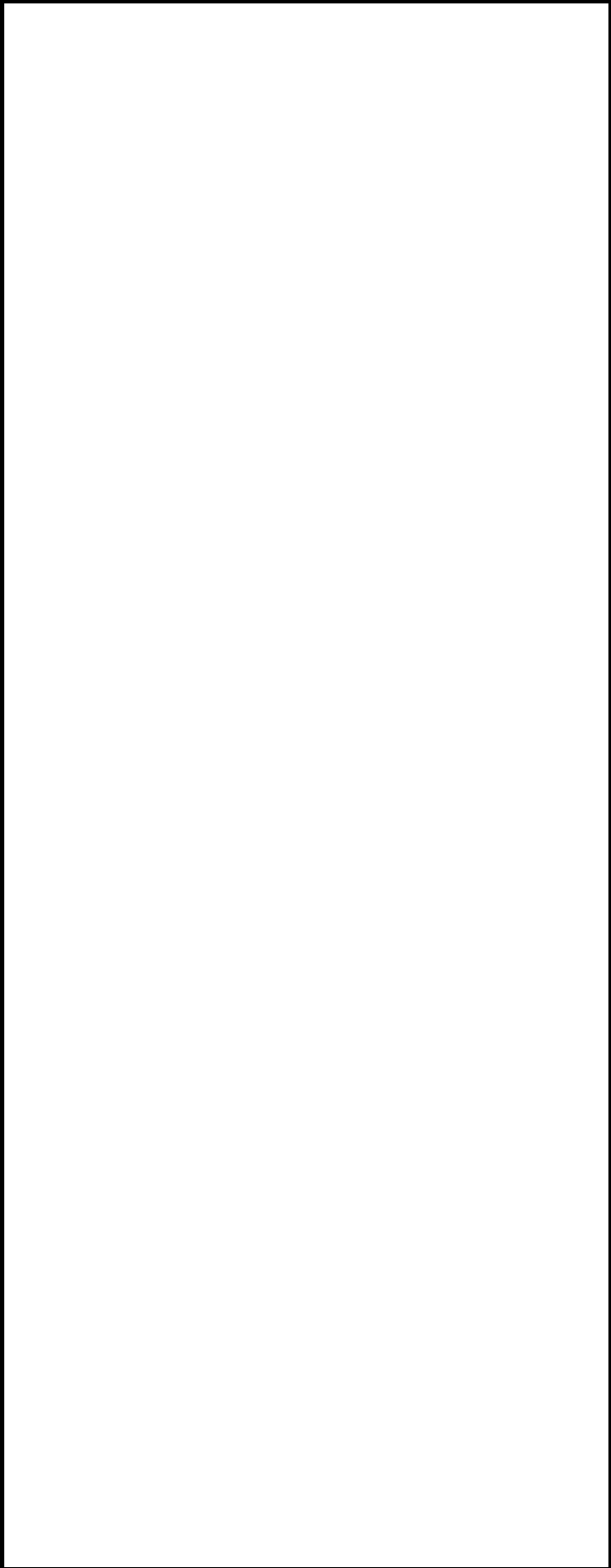


FIGURE 37 : LE PIEGE (VIDEO)



FIGURE 37 : SUITE

FIGURE 37 : SUITE

FIGURE 37 : SUITE

FIGURE 37 : SUITE

FIGURE 37 : SUITE

FIGURE 37 : SUITE

FIGURE 37 : SUITE

FIGURE 37 : SUITE

FIGURE 37 : SUITE

```

5 REM *** CHARGEUR - VERSION VIDEO ***
10 CLEAR 50,&H7F0: DEFINT A-Z
20 INIT#1,"CASI:"
30 INPUT#1,N$,D,F
40 MOTOR
50 PRINT"Trouv :";N$
60 FOR I=D-1 TO F
70 POKE I,INP(#1)
80 NEXT: MOTOR
90 END
100 CLEAR 50,&H7F0
110 D=&H800: F=&H18E8
120 N$="AVENT": INIT#1,"CASO:"
130 INPUT"Magnto OK";T$
140 PRINT#1,N$,D,F: MOTOR
150 FOR I=1 TO 1800: NEXT
160 FOR I=D TO F: OUT#1,PEEK(I)
170 NEXT: MOTOR
180 END

```

```

5 REM *** ENTREUR DE CODES - VIDEO ***
10 CLEAR 50,&H7FF:A=&H800
20 PRINTHEX$(A);" : " ;:INPUT C$
30 V=VAL("&H"+C$):POKE A,V
49 A=A+1:IF A>&H18E8 THEN
PRINT"...":BEEP 2,3:END
50 GOTO20

```

FIGURE 37 : FIN

FIGURE 38 : CHARGEURS

Othello-Reversi

Remarquable à la fois par la simplicité de ses règles et par la complexité des tactiques et des stratégies à mettre en œuvre, Othello-Reversi a conquis en quelques années des dizaines de millions de passionnés dans le monde. Bien qu'il reste relativement peu pratiqué en France, il possède toutes les qualités d'un jeu exemplaire.

Le Reversi a été inventé dans la seconde moitié du 19^{ème} siècle en Angleterre. Après de nombreuses aventures, un Japonais l'a réinventé sous le nom d'Othello.

Les règles du jeu sont simples. Othello se joue sur un plateau 8*8 dont toutes les cases sont de la même couleur . Les pions utilisés sont à double face d'un côté blanc, de l'autre côté noir. Chaque joueur prend une couleur, l'un sera blanc, l'autre noir. Au début de la partie, les joueurs disposent quatre pions au centre de l'échiquier.

Les premiers pions posés, les joueurs jouent chacun leur tour en posant un pion de leur couleur sur une case libre, de telle façon qu'entre ce pion posé et un autre pion de la même couleur, un pion au moins de la couleur adverse soit pris en tenaille, selon une ligne horizontale, verticale ou diagonale. Le ou les pions pris sont alors retournés et ils prennent la couleur du preneur, d'où le nom de Reversi. La pose d'un pion peut prendre en tenaille des pions selon plusieurs lignes. Ils sont alors tous retournés. Mais il n'y a pas transitivité, un pion retourné ne peut entraîner de nouveaux retournements.

Il y a obligation de toujours retourner un pion adverse. Lorsqu'un joueur ne peut pas jouer de coup qui le lui permette, il doit passer son tour. Lorsqu'aucun joueur ne peut plus jouer, soit que toutes les cases soient remplies, soit qu'il n'y ait plus de coup possible selon la règle ci-dessus, la partie s'arrête et le vainqueur est celui dont la couleur apparaît sur le plus grand nombre de pions posés.

Sur le CANON X-07, le logiciel suit scrupuleusement les règles décrites ci-dessus. Les pions ne pouvant être colorés, vous posséderez des pions de forme différente au programme qui se révèle, soit dit en passant, très fort.

Le mode d'emploi est le suivant :

- Après avoir chargé le programme en mémoire grâce à la K7 (CLOAD puis RUN pour charger les codes) ou à la "main" grâce aux chargeurs joints, vous devez lancer le programme par EXEC &HE00 suivi de RETURN.
- Une présentation défile. Tapez sur RETURN, le X-07 vous demande si vous désirez jouer contre lui ou contre un adversaire humain.
- Ensuite , le niveau de jeu vous est demandé, entre 1 et 9 . Le niveau 1 est beaucoup plus facile que le 9.
- Le X-07 s'enquiert enfin de vos initiales (4 lettres), tapez-les et le CANON vous présentera le damier

- Sur la gauche de l'écran, l'échiquier 8*8 est dessiné et à droite de l'afficheur le nombre de pions gagnés par chacun des deux joueurs.
- Pour jouer, vous devez déplacer le point clignotant avec les quatre curseurs. Dès que vous avez choisi votre place, tapez sur RETURN et le X-07 calculera et affichera les pions gagnés. Le X-07 jouera ensuite et le jeu continuera ainsi jusqu'à la fin de la partie.
- Notons que si vous devez passer, vous pouvez appuyer sur "P", le X-07 vous accordera ce droit si vous êtes vraiment bloqué. Sinon, il vous affichera vos possibilités de jeu. De plus, le jeu peut être quitté à tout moment par l'habituel "CTRL Q".

Bonne chance. Dommage, on ne peut pas tricher.

Longueur de REVERSI : 3073 octets

Implantation de REVERSI : de 800h à 1400h

FIGURE 39 : OTHELLO

FIGURE 39 : SUITE

FIGURE 39 : SUITE

FIGURE 39 : SUITE

FIGURE 39 : SUITE

FIGURE 39 : SUITE

FIGURE 39 : SUITE

FIGURE 39 : SUITE


```

5 REM *** OTHELLO - REVERSI ***
10 CLEAR 50,&H7FF
20 INIT#1,"CASI:"
30 INPUT#1,N$,D,F
40 MOTOR
50 PRINT"Trouv :";N$
60 FOR I=D-1 TO F
70 POKE I,INP(#1)
80 NEXT: MOTOR
90 END

100 CLEAR 50,&H7FF
110 D=&H800: F=&H1400
120 N$="OTHEL": INIT#1,"CASO:"
130 INPUT"Magnto OK";T$
140 PRINT#1,N$,D,F: MOTOR
150 FOR I=1 TO 1800: NEXT
160 FOR I=D TO F: OUT#1,PEEK(I)
170 NEXT: MOTOR
180 END


5 REM *** ENTREUR DE CODES - VIDEO ***
10 CLEAR 50,&H7FF:A=&H800
20 PRINTEX$(A);" : ";;INPUT C$
30 V=VAL("&H"+C$):POKE A,V
49 A=A+1:IF A>&H1400 THEN
PRINT"...":BEEP 2,3:END
50 GOTO20

```

FIGURE 40 : CHARGEURS

Conclusion

En l'espace de six mois, nous avons concocté deux ouvrages sur le CANON X-07 , très demandés par les utilisateurs de cette superbe machine.

Nous les avons composés avec grand plaisir et nous espérons qu'ils auront apporté les réponses nécessaires aux nombreux problèmes auxquels se trouvent confrontés quotidiennement les canonistes.

Bien entendu, tout renseignement sur tel ou tel sujet traité dans "Les mystères du X-07" ou dans le présent ouvrage vous sera immédiatement délivré par le service technique du CLUB C7, par courrier, par téléphone, de vive voix ou par M INITEL.

Les Editions NEPTUNE publieront régulièrement de nouveaux ouvrages informatiques en essayant toujours de garder une optique de bas prix d'originalité et de haute qualité alliée à une simplicité de présentation.

N'hésitez surtout pas à nous contacter, le marché de l'informatique évolue rapidement et nos prochaines publications seront éditées avec l'intention certaine de rester à la pointe de l'actualité.

Bravo au X-07 qui nous a tellement étonné et qui nous surprendra sans doute encore longtemps.

Le directeur de la publication

André TONIC